HTML وCSS · اليوم 1 · الأسبوع 1 · ساعتان · دورة أساسيات البرمجة

مدخل إلى البرمجة وأول صفحة HTML

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يشرح الفرق بين HTML (لغة ترميز) وC# (لغة برمجة) وSQL (لغة استعلام) ودور كل منها في التطبيق.
- ينشئ ملف index.html في Visual Studio Community ويفتحه في المتصفح.
- يكتب الهيكل الأساسي لصفحة HTML.
- يستخدم العناوين h1–h6 والفقرات p والتنسيق strong وem.
- يضيف روابط a وصورًا img بشكل صحيح.

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. ما هي البرمجة؟

البرمجة هي كتابة **تعليمات دقيقة ومرتبة** ينفذها الحاسوب خطوة بخطوة. الحاسوب لا يخمّن، ينفّذ ما كُتب بالضبط.

في هذه الدورة سنتعلم ثلاث لغات تعمل معًا في أي تطبيق حقيقي:

- **HTML وCSS**: الواجهة التي يراها المستخدم (الشكل والمحتوى).
- **C#**: المنطق والحسابات واتخاذ القرارات.
- **SQL Server**: حفظ البيانات واسترجاعها.

> مثال من الواقع: في نظام تسجيل الطلاب، صفحة التسجيل = HTML، التحقق من البيانات وحساب المعدل = C#، حفظ بيانات الطالب = SQL Server.

2. تجهيز الأدوات

سنكتب HTML في **Visual Studio Community** المثبت على أجهزتكم، ونشاهد النتيجة في المتصفح (Chrome أو Edge).

1. أنشئ مجلدًا باسم `my-site` على سطح المكتب.
2. في Visual Studio اختر **File** ← **Open** ← **Folder** ثم المجلد `my-site` .
3. في **Solution Explorer** انقر بالزر الأيمن على المجلد ← **New File** ← **Add** واكتب الاسم `index.html` .
4. اكتب الكود واحفظ بـ `Ctrl+S` ، ثم افتح الملف في المتصفح بالنقر المزدوج عليه من المجلد. بعد كل تعديل: احفظ ثم حدّث المتصفح بـ `F5` .

Visual Studio يقترح أسماء الوسوم والخصائص أثناء الكتابة. وللتجربة السريعة استخدم «محرر HTML» في هذه المنصة.

> إذا لم تظهر اقتراحات HTML في Visual Studio، افتح Modify ← Visual Studio Installer وتأكد من تثبيت حزمة ASP.NET and web development.

3. هيكل صفحة HTML

كل صفحة تتكون من **وسوم** (Tags) تُكتب بين علامتي `< >` ، وأغلب الوسوم لها وسم فتح ووسم إغلاق.

```HTML
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>صفحتي الأولى</title>
</head>
<body>
  <h1>مرحبًا بالعالم</h1>
  <p>هذه أول صفحة أكتبها بلغة HTML.</p>
</body>
</html>
```

> **ماذا يفعل هذا الكود؟** هذه صفحة HTML كاملة بأبسط شكل. عند فتحها في المتصفح سترى عنوانًا كبيرًا «مرحبًا بالعالم» وتحته فقرة قصيرة، وسيظهر «صفحتي الأولى» في أعلى تبويب المتصفح.

`<!DOCTYPE html>`	هذا السطر يخبر المتصفح أن الملف مكتوب بالإصدار الحديث من HTML (وهو HTML5). يُكتب دائمًا في أول سطر من الصفحة ولا يظهر على الشاشة.
`<html lang="ar" dir="rtl">`	الوسم html هو الوعاء الذي يحتوي الصفحة كلها. الخاصية "lang="ar تقول إن لغة الصفحة عربية، والخاصية "dir="rtl تجعل اتجاه الكتابة من اليمين إلى اليسار كما يناسب العربية.
`<head>`	الوسم head هو «رأس» الصفحة، ويضم معلومات عن الصفحة لا تظهر داخلها مباشرة، مثل الترميز والعنوان الذي يظهر في التبويب.

`<meta charset="UTF-8">`	الوسم meta مع الخاصية charset="UTF-8" يحدد طريقة ترميز الحروف، أي كيف تُحفظ الحروف داخل الملف. الترميز UTF-8 يدعم الحروف العربية وغيرها، فلا تظهر الكلمات رموزًا غريبة.
`<title>صفحتي الأولى</title>`	الوسم title يحدد عنوان الصفحة الذي يظهر في أعلى تبويب المتصفح، وليس داخل الصفحة نفسها.
`</head>`	هذا وسم إغلاق؛ الشرطة المائلة / تعني نهاية قسم head. معظم وسوم HTML تُفتح ثم تُغلق بهذا الشكل.
`<body>`	الوسم body هو «جسم» الصفحة، وكل ما نكتبه داخله هو ما يراه الزائر فعلًا على الشاشة.
`<h1>مرحبًا بالعالم</h1>`	الوسم h1 يعرض عنوانًا رئيسيًا بخط كبير وعريض. هنا سيظهر النص «مرحبًا بالعالم» كأكبر عنوان في الصفحة.
`<p>هذه أول صفحة أكتبها بلغة HTML.</p>`	الوسم p اختصار لكلمة paragraph أي فقرة، ويعرض نصًا عاديًا. سيظهر هذا النص تحت العنوان بخط عادي.
`</body>`	هذا السطر والذي بعده يغلقان الوسمين body ثم html، أي أن جسم الصفحة انتهى ثم الصفحة كلها انتهت.

head يحتوي معلومات عن الصفحة لا تظهر فيها، وbody يحتوي كل ما يراه المستخدم. dir="rtl" يجعل الاتجاه من اليمين لليسار.

4. العناوين والفقرات والتنسيق

العناوين من `h1` (الأكبر) إلى `h6` (الأصغر). الفقرة `p` ، وسطر جديد `br` ، والخط العريض `strong` ، والمائل `em` .

HTML

```
<h1>عنوان رئيسي</h1>
<h2>عنوان فرعي</h2>
<p>هذه فقرة فيها كلمة <strong>مهمة</strong> وكلمة <em>مائلة</em>.</p>
<p>سطر أول<br>سطر ثانٍ</p>
```

ماذا يفعل هذا الكود؟ هذا المثال يعرض أنواعًا من النصوص: عنوانًا رئيسيًا، وعنوانًا فرعيًا أصغر، وفقرة فيها كلمة عريضة وكلمة مائلة، ثم فقرة مقسومة على سطرين.

`<h1>عنوان رئيسي</h1>`	الوسم h1 يعرض أهم وأكبر عنوان في الصفحة. توجد عناوين من h1 (الأكبر) إلى h6 (الأصغر).
`<h2>عنوان فرعي</h2>`	الوسم h2 عنوان من المستوى الثاني، فيظهر أصغر قليلًا من h1، ونستخدمه لعناوين الأقسام الفرعية.

هذه فقرة فيها كلمة مهمة وكلمة مائلة.	هذه فقرة p بداخلها وسمان: الوسم strong يجعل كلمة «مهمة» عريضة ويعني أنها مهمة، والوسم em يجعل كلمة «مائلة» مائلة ويعني التوكيد عليها. لاحظ أن الوسوم يمكن أن توضع داخل بعضها.
سطر أول سطر ثانٍ	الوسم br ينقل الكتابة إلى سطر جديد داخل الفقرة نفسها، فيظهر «سطر ثانٍ» تحت «سطر أول». هذا الوسم ليس له وسم إغلاق لأنه لا يحتوي على نص.

5. الروابط والصور

الرابط يستخدم الخاصية `href` ، والصورة تستخدم `src` للمسار و `alt` لوصف الصورة. الصورة وسم مفرد لا يُغلق.

```HTML
<a href="https://www.google.com" target="_blank">افتح جوجل</a>

<img src="https://placehold.co/300x150" alt="صورة توضيحية" width="300">
```

ماذا يفعل هذا الكود؟ هذا المثال يضيف رابطًا يفتح موقع جوجل في تبويب جديد، وصورة عرضها 300 بكسل.

افتح جوجل	الوسم a يصنع رابطًا يمكن الضغط عليه، والخاصية href تحدد عنوان الموقع الذي ينتقل إليه. الخاصية target="_blank" تجعل الرابط يُفتح في تبويب جديد، والنص «افتح جوجل» هو ما يظهر للزائر.
<img src="https://placehold.co/300x150" alt="صورة توضيحية" width="300">	الوسم img يعرض صورة، والخاصية src تحدد مكان الصورة (هنا صورة تجريبية من الإنترنت). الخاصية alt نص بديل يظهر إذا لم تُحمَّل الصورة وتقرؤه برامج المكفوفين، والخاصية width تجعل عرض الصورة 300 بكسل. الوسم img ليس له وسم إغلاق.

target="_blank" يفتح الرابط في نافذة جديدة. احفظ الصور داخل مجلد المشروع واكتب اسمها: src="photo.jpg".

التمارين العملية

تمرين 1: صفحة تعريف بنفسي

سهل

أنشئ صفحة فيها عنوان رئيسي باسمك، وعنوان فرعي «نبذة عني»، وفقرتان عن نفسك وهواياتك، واجعل كلمة واحدة على الأقل بخط عريض.

الحل المقترح:

```HTML
<!DOCTYPE html>
<html lang="ar" dir="rtl">
```

```
<head><meta charset="UTF-8"><title>أحمد</title></head>
<body>
  <h1>أحمد علي</h1>
  <h2>نبذة عني</h2>
  <p>أنا طالب أحب <strong>البرمجة</strong> والتقنية.</p>
  <p>هواياتي: القراءة وكرة القدم.</p>
</body>
</html>
```

ماذا يفعل هذا الكود؟ صفحة تعريف شخصية بسيطة: تعرض اسم «أحمد علي» كعنوان كبير، ثم عنوانًا فرعيًا «نبذة عني»، ثم فقرتين عنه.

`<!DOCTYPE html>`	يخبر المتصفح أن الصفحة مكتوبة بلغة HTML الحديثة، ويوضع دائمًا في أول سطر.
`<html lang="ar" dir="rtl">`	الوسم html يحتوي الصفحة كلها؛ lang="ar" يحدد أن اللغة عربية، وdir="rtl" يجعل الاتجاه من اليمين إلى اليسار.
`<head><meta charset="UTF-8">` `<title>أحمد</title></head>`	هنا كُتب قسم head كاملًا في سطر واحد: الوسم meta charset="UTF-8" لدعم الحروف العربية، والوسم title يجعل كلمة «أحمد» تظهر في تبويب المتصفح.
`<body>`	بداية جسم الصفحة؛ كل ما بداخله يظهر على الشاشة.
`<h1>أحمد علي</h1>`	الوسم h1 يعرض الاسم «أحمد علي» كعنوان رئيسي كبير في أعلى الصفحة.
`<h2>نبذة عني</h2>`	الوسم h2 عنوان فرعي أصغر من h1، يبدأ قسم التعريف بالنفس.

`<p>أنا طالب أحب`
`<strong>البرمجة</strong> والتقنية.`
`</p>`
	فقرة p تعرض جملة تعريفية، والوسم strong يجعل كلمة «البرمجة» عريضة للتأكيد على أهميتها.

`<p>هواياتي: القراءة وكرة القدم.</p>`	فقرة ثانية تعرض الهوايات، وتظهر تحت الفقرة الأولى في سطر جديد تلقائيًا لأن كل فقرة p تبدأ من سطر جديد.
`</body>`	هذا السطر والذي بعده يغلقان جسم الصفحة ثم الصفحة كلها.

تمرين 2: صورة ورابط

سهل

أضف إلى صفحتك صورة بعرض 250 بكسل مع نص بديل، ورابطًا لموقع تحبه يفتح في نافذة جديدة.

الحل المقترح:

HTML

```
<img src="https://placehold.co/250x250" alt="صورتي" width="250">
```

```
<p>موقعي المفضل: <a href="https://www.wikipedia.org" target="_blank">ويكيبيديا</a></p>
```

ماذا يفعل هذا الكود؟ يعرض هذا الكود صورة مربعة بحجم 250 بكسل، وتحتها جملة فيها رابط إلى موقع ويكيبيديا يُفتح في تبويب جديد.

`<img src="https://placehold.co/250x250" alt="صورتي" width="250">`	الوسم img يعرض صورة مصدرها في الخاصية src، والخاصية alt تعطي وصفًا نصيًا «صورتي» يظهر إذا تعذر تحميل الصورة، والخاصية width تحدد عرضها بـ 250 بكسل.
`<p>موقعي المفضل: <a href="https://www.wikipedia.org" target="_blank">ويكيبيديا</a></p>`	فقرة p بداخلها رابط a؛ الخاصية href تحدد عنوان موقع ويكيبيديا، وtarget="_blank" تفتحه في تبويب جديد. كلمة «ويكيبيديا» تظهر بلون مختلف وتحتها خط لتدل على أنها رابط.

تمرين 3: تحدٍّ: صفحة وصفة طعام

تحدٍّ

صمم صفحة لوصفة طعام: عنوان رئيسي باسم الوصفة، صورة، عنوانان فرعيان «المكونات» و«طريقة التحضير» وتحت كل منهما فقرة.

الحل المقترح:

HTML

```
<h1>الكبسة</h1>
<img src="https://placehold.co/400x200" alt="صورة الكبسة" width="400">
<h2>المكونات</h2>
<p>أرز، دجاج، بصل، طماطم، بهارات الكبسة.</p>
<h2>طريقة التحضير</h2>
<p>نقلي البصل ثم نضيف الدجاج والطماطم والبهارات، ثم الماء والأرز ونتركه ينضج.</p>
```

ماذا يفعل هذا الكود؟ صفحة وصفة طبخ للكبسة: عنوان رئيسي، ثم صورة، ثم قسم للمكونات وقسم لطريقة التحضير، لكل قسم عنوان فرعي وفقرة.

`<h1>الكبسة</h1>`	الوسم h1 يعرض اسم الوصفة «الكبسة» كعنوان رئيسي كبير.
`<img src="https://placehold.co/400x200" alt="صورة الكبسة" width="400">`	الوسم img يعرض صورة للوصفة؛ src مكان الصورة، وalt وصف نصي بديل، وwidth يجعل عرضها 400 بكسل.
`<h2>المكونات</h2>`	الوسم h2 عنوان فرعي يبدأ قسم المكونات، ويظهر أصغر من العنوان الرئيسي.
`<p>أرز، دجاج، بصل، طماطم، بهارات الكبسة.</p>`	فقرة p تعرض قائمة المكونات كنص عادي.

`<h2>طريقة التحضير</h2>`	عنوان فرعي آخر من نوع h2 يبدأ قسم طريقة التحضير، فتبدو الصفحة منظمة في أقسام.
`<p>نقلي البصل ثم نضيف الدجاج والطماطم والبهارات، ثم الماء والأرز ونتركه ينضج.</p>`	فقرة p تشرح خطوات التحضير، وتظهر تحت عنوان «طريقة التحضير».

اختبر نفسك (3 أسئلة)

1. أي لغة مسؤولة عن حفظ البيانات واسترجاعها؟

HTML | C# | SQL

2. أين نكتب المحتوى الذي يظهر للمستخدم؟

داخل head | داخل body | داخل title

3. ما الخاصية التي تحدد مسار الصورة؟

href | src | alt

الإجابات: 1) SQL · 2) داخل body · 3) src

الواجب: أكمل صفحة التعريف وأضف إليها رابطين وصورتين.

HTML وCSS اليوم 2 · الأسبوع 1 · ساعتان · دورة أساسيات البرمجة

القوائم والجداول والنماذج

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- ينشئ قوائم مرقمة ol وغير مرقمة ul.
- يبني جدولًا بصفوف وأعمدة وعناوين.
- يصمم نموذج إدخال بأنواع حقول مختلفة.
- يقسّم الصفحة بوسوم header وmain وfooter.

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. القوائم

`ul` قائمة نقطية، و `ol` قائمة مرقمة، وكل عنصر يوضع داخل `li` .

HTML

```
<h3>أدوات الدورة</h3>
<ul>
  <li>Visual Studio Community</li>
  <li>المتصفح</li>
  <li>SQL Server</li>
</ul>

<h3>خطوات حفظ الملف</h3>
<ol>
  <li>اكتب الكود</li>
  <li>اضغط Ctrl + S</li>
  <li>حدّث المتصفح</li>
</ol>
```

ماذا يفعل هذا الكود؟ يعرض هذا المثال نوعين من القوائم: قائمة نقطية (بدون ترتيب) لأدوات الدورة، وقائمة مرقمة لخطوات حفظ الملف.

`<h3>أدوات الدورة</h3>`	الوسم h3 عنوان من المستوى الثالث، أصغر من h1 وh2، ويظهر فوق القائمة الأولى.
`<ul>`	الوسم ul اختصار unordered list أي «قائمة غير مرتبة»، وتظهر عناصرها مع نقطة صغيرة أمام كل عنصر بدل الأرقام.
`<li>Visual Studio Community</li>`	الوسم li اختصار list item أي «عنصر في القائمة». كل li يظهر في سطر خاص به مع نقطة أمامه، والسطران التاليان عنصران آخران بنفس الطريقة.
`</ul>`	يغلق القائمة غير المرتبة؛ كل عناصر li يجب أن تكون بين وسم الفتح ووسم الإغلاق.
`<h3>خطوات حفظ الملف</h3>`	عنوان h3 ثانٍ يبدأ القائمة الثانية.
`<ol>`	الوسم ol اختصار ordered list أي «قائمة مرتبة»، فيضع المتصفح أرقامًا تلقائيًا أمام العناصر: 1، 2، 3. نستخدمها عندما يكون الترتيب مهمًا مثل الخطوات.
`<li>اكتب الكود</li>`	أول عنصر في القائمة المرتبة، سيظهر بجانبه الرقم 1، ثم يأتي العنصران التاليان بالرقمين 2 و3 تلقائيًا دون أن نكتب الأرقام بأنفسنا.
`</ol>`	يغلق القائمة المرتبة.

2. الجداول

الجدول `table` يتكون من صفوف `tr` ، وكل صف فيه خلايا عنوان `th` أو خلايا بيانات `td` .

HTML

```
<table border="1">
  <tr>
    <th>الاسم</th>
    <th>الدرجة</th>
  </tr>
  <tr>
    <td>سارة</td>
    <td>95</td>
  </tr>
  <tr>
    <td>خالد</td>
    <td>88</td>
  </tr>
</table>
```

ماذا يفعل هذا الكود؟ يعرض هذا الكود جدولًا صغيرًا بإطار، فيه صف عناوين (الاسم، الدرجة) وتحته صفان لطالبين ودرجاتهما.

`<table border="1">`	الوسم table يبدأ جدولًا. الخاصية "border="1 ترسم خطوطًا حول الخلايا حتى يظهر الجدول بإطار؛ وهي طريقة قديمة سهلة، وفي الدروس القادمة سنرسم الإطارات باستخدام CSS.
`<tr>`	الوسم tr اختصار table row أي «صف في الجدول». كل ما بين <tr> و</tr> يظهر في صف أفقي واحد.
`<th>الاسم</th>`	الوسم th اختصار table header أي «خلية عنوان»، ويظهر نصها عريضًا وفي الوسط عادةً. هذه الخلية والتي تليها تكوّنان صف العناوين.
`</tr>`	يغلق الصف الحالي، وبعده يبدأ صف جديد في سطر جديد من الجدول.
`<td>سارة</td>`	الوسم td اختصار table data أي «خلية بيانات» عادية. هنا تظهر «سارة» تحت عمود الاسم، والسطر التالي يضع درجتها 95 تحت عمود الدرجة.
`<td>خالد</td>`	صف ثالث بنفس الطريقة: خلية لاسم «خالد» وخلية لدرجته 88.
`</table>`	يغلق الجدول كاملًا.

3. النماذج (Forms)

النموذج `form` يجمع بيانات المستخدم. الحقل `input` يتغير شكله حسب `type`، و`label` يصف الحقل، و`select` قائمة منسدلة، و`textarea` نص طويل.

HTML

```
<form>
  <label>الاسم*: /label>
  <input type="text" placeholder="اكتب اسمك" br><br>
```

```
  <label>البريد*: /label>
  <input type="email"><br><br>

  <label>العمر*: /label>
  <input type="number" min="15" max="60"><br><br>

  <label>الجنس*: /label>
  <input type="radio" name="g"> ذكر
  <input type="radio" name="g"> أنثى<br><br>

  <label>الدورة*: /label>
  <select>
    <option>HTML</option>
    <option>C#</option>
  </select><br><br>

  <input type="checkbox"> أوافق على الشروط<br><br>
  <button type="submit">تسجيل</button>
</form>
```

ماذا يفعل هذا الكود؟ يعرض هذا الكود نموذج تسجيل (فورم) فيه خانات لكتابة الاسم والبريد والعمر، واختيار الجنس والدورة، ومربع موافقة، وزر «تسجيل».

`<form>`	الوسم form يصنع نموذجًا، أي مجموعة خانات يملؤها المستخدم ثم يرسلها. كل عناصر الإدخال توضع داخله.
`<label>الاسم*: /label>`	الوسم label يعرض نصًا وصفيًا بجانب الخانة ليعرف المستخدم ماذا يكتب فيها.
`<input type="text" placeholder="اكتب اسمك"> br> `	الوسم input يصنع خانة إدخال، و"type="text تعني أنها لكتابة نص عادي. الخاصية placeholder تعرض نصًا باهتًا داخل الخانة يختفي عند الكتابة، والوسمان br يتركان سطرًا فارغًا قبل الخانة التالية.
`<label>البريد*: /label>`	نص وصفي لخانة البريد الإلكتروني.
`<input type="email">  `	خانة من نوع email مخصصة للبريد الإلكتروني؛ عند الإرسال يتحقق المتصفح أن النص المكتوب يشبه بريدًا صحيحًا (فيه @ مثلًا).
`<label>العمر*: /label>`	نص وصفي لخانة العمر.
`<input type="number" min="15" max="60">  `	خانة من نوع number لا تقبل إلا الأرقام، وتظهر بجانبها أسهم صغيرة للزيادة والنقصان. الخاصيتان min وmax تحددان أصغر قيمة مسموحة (15) وأكبر قيمة (60).
`<label>الجنس*: /label>`	نص وصفي لخيارات الجنس.

`<input type="radio" name="g"> ذكر`	النوع radio يصنع دائرة اختيار صغيرة. هذا السطر والذي بعده لهما نفس الاسم name="g"، لذلك يعتبرهما المتصفح مجموعة واحدة ولا يسمح إلا باختيار واحد منهما: ذكر أو أنثى.
`<label>الدورة: /label>`	نص وصفي لقائمة اختيار الدورة.
`<select>`	الوسم select يصنع قائمة منسدلة، أي مربعًا تضغط عليه فتظهر خيارات تختار منها واحدًا.
`<option>HTML</option>`	الوسم option يمثل خيارًا واحدًا داخل القائمة المنسدلة. هنا خياران: HTML وفي السطر التالي C#.
`</select>  `	يغلق القائمة المنسدلة، ثم يترك سطرًا فارغًا بالوسمين br.
`<input type="checkbox"> أوافق على الشروط  `	النوع checkbox يصنع مربعًا صغيرًا يمكن تحديده بعلامة صح أو إلغاء تحديده، وبجانبه النص «أوافق على الشروط».
`<button type="submit">تسجيل</button>`	الوسم button يصنع زرًا مكتوبًا عليه «تسجيل»، وtype="submit" تعني أن الضغط عليه يرسل النموذج. بما أننا لم نحدد إلى أين يُرسل، ستُعاد تحميل الصفحة نفسها فقط.
`</form>`	يغلق النموذج؛ كل الخانات التي بين وسم الفتح والإغلاق تنتمي لهذا النموذج.

أزرار radio التي لها نفس name تسمح باختيار واحد فقط من بينها.

4. تقسيم الصفحة

نستخدم وسومًا ذات معنى: `header` لرأس الصفحة، `nav` للقائمة، `main` للمحتوى، `footer` للتذييل، و `div` كصندوق عام للتجميع.

HTML

```
<header><h1>معهد البرمجة</h1></header>
<nav><a href="#">الرئيسية</a> | <a href="#">الدورات</a></nav>
<main>
  <p>مرحبًا بكم في دورة أساسيات البرمجة.</p>
</main>
<footer><p>جميع الحقوق محفوظة 2026</p></footer>
```

ماذا يفعل هذا الكود؟ يقسم هذا الكود الصفحة إلى أجزاء ذات معنى: رأس فيه اسم المعهد، وشريط روابط، ومحتوى رئيسي، وتذييل في الأسفل لحقوق النشر.

`<header><h1>معهد البرمجة</h1></header>`	الوسم header يمثل رأس الصفحة أو الجزء العلوي منها، وبداخله عنوان h1 باسم المعهد. هذه الوسوم تسمى «دلالية» لأنها توضح معنى كل جزء دون أن تغير شكله كثيرًا.
`<nav><a href="#">الرئيسية</a> | <a href="#">الدورات</a></nav>`	الوسم nav اختصار navigation ويضم روابط التنقل في الموقع. بداخله رابطان a، والقيمة href="#" تعني رابطًا مؤقتًا لا ينقل إلى صفحة أخرى، والعلامة | مجرد نص يفصل بين الرابطين.
`<main>`	الوسم main يضم المحتوى الرئيسي والأهم في الصفحة.
`<p>مرحبًا بكم في دورة أساسيات البرمجة.</p>`	فقرة ترحيبية داخل المحتوى الرئيسي.
`</main>`	يغلق قسم المحتوى الرئيسي.
`<footer><p>جميع الحقوق محفوظة 2026</p></footer>`	الوسم footer يمثل تذييل الصفحة، أي الجزء السفلي الذي يحتوي عادة على حقوق النشر ومعلومات التواصل.

التمارين العملية

تمرين 1: قائمة مهامي

سهل

أنشئ قائمة مرقمة بخمس مهام يومية، وقائمة نقطية بثلاث لغات برمجة تعرفها.

الحل المقترح:

```HTML
<h2>مهامي اليوم</h2>
<ol>
  <li>حضور الدورة</li><li>حل التمارين</li><li>مراجعة الدرس</li><li>الرياضة</li><li>القراءة</li>
</ol>
<h2>لغات أعرفها</h2>
<ul><li>HTML</li><li>C#</li><li>SQL</li></ul>
```

ماذا يفعل هذا الكود؟ يعرض هذا الحل قائمة مرقمة بخمس مهام لليوم، وتحتها قائمة نقطية باللغات التي يعرفها الطالب.

`<h2>مهامي اليوم</h2>`	عنوان فرعي h2 يظهر فوق قائمة المهام.
`<ol>`	الوسم ol يبدأ قائمة مرتبة، فتظهر المهام مرقمة تلقائيًا من 1 إلى 5.
`<li>حضور الدورة</li><li>حل التمارين</li><li>مراجعة الدرس</li><li>الرياضة</li><li>القراءة</li>`	خمسة عناصر li مكتوبة في سطر واحد. رغم ذلك سيعرض المتصفح كل عنصر في سطر مستقل برقمه، لأن شكل الكود لا يؤثر على العرض بل الوسوم هي التي تحدده.

`</ol>`	يغلق القائمة المرتبة.
`<h2>لغات أعرفها</h2>`	عنوان فرعي ثانٍ فوق قائمة اللغات.
`<ul><li>HTML</li><li>C#</li><li>SQL</li></ul>`	قائمة غير مرتبة ul كاملة في سطر واحد، فيها ثلاثة عناصر li، وستظهر كل لغة في سطر مع نقطة أمامها.

تمرين 2: جدول درجات

متوسط

صمم جدولًا لثلاثة طلاب بأعمدة: الرقم، الاسم، HTML، C#، المجموع.

الحل المقترح:

```HTML
<table border="1" cellpadding="8">
  <tr><th>#</th><th>الاسم</th><th>HTML</th><th>C#</th><th>المجموع</th></tr>
  <tr><td>1</td><td>سارة</td><td>45</td><td>48</td><td>93</td></tr>
  <tr><td>2</td><td>خالد</td><td>40</td><td>42</td><td>82</td></tr>
  <tr><td>3</td><td>مريم</td><td>47</td><td>44</td><td>91</td></tr>
</table>
```

ماذا يفعل هذا الكود؟ يعرض هذا الحل جدول درجات بإطار، فيه صف عناوين وثلاثة طلاب مع درجاتهم في HTML وC# ومجموعهم.

`<table border="1" cellpadding="8">`	الوسم table يبدأ الجدول؛ "border="1 يرسم خطوطًا حول الخلايا، و"cellpadding="8 يضع مسافة 8 بكسل بين نص الخلية وحدودها حتى لا يبدو مزدحمًا. هاتان خاصيتان قديمتان، ولاحقًا نستخدم CSS بدلًا منهما.
`<tr><th>#</th><th>الاسم</th><th>HTML</th><th>C#</th><th>المجموع</th></tr>`	صف tr كامل في سطر واحد يحتوي خمس خلايا عنوان th، فتظهر عناوين الأعمدة بخط عريض في أعلى الجدول.
`<tr><td>1</td><td>سارة</td><td>45</td><td>48</td><td>93</td></tr>`	صف بيانات يحتوي خمس خلايا td: رقم الطالب واسمه ودرجاته ومجموعه، وكل خلية تقع تحت العنوان المقابل لها.
`<tr><td>2</td><td>خالد</td><td>40</td><td>42</td><td>82</td></tr>`	صف بيانات الطالب الثاني بنفس الترتيب. والسطر التالي صف الطالبة الثالثة مريم بنفس الطريقة.
`</table>`	يغلق الجدول.

تمرين 3: نموذج تسجيل طالب

تحدٍّ

صمم نموذج تسجيل فيه: الاسم الكامل، البريد، رقم الهاتف، العمر، الجنس (radio)، الدورة (select)، ملاحظات (textarea)، وزر «تسجيل». هذا النموذج سيدخل في مشروعك النهائي.

الحل المقترح:

HTML

```
<h2>تسجيل طالب جديد</h2>
<form>
  <p><label>الاسم الكامل: <input type="text" required></label></p>
  <p><label>البريد: <input type="email"></label></p>
  <p><label>الهاتف: <input type="tel"></label></p>
  <p><label>العمر: <input type="number" min="15"></label></p>
  <p>الجنس:
    <label><input type="radio" name="g"> ذكر</label>
    <label><input type="radio" name="g"> أنثى</label></p>
  <p><label>الدورة:
    <select><option>HTML</option><option>C#</option><option>SQL</option></select>
</label></p>
  <p><label>ملاحظات: br><textarea rows="3" cols="30"></textarea></label></p>
  <button type="submit">تسجيل</button>
</form>
```

ماذا يفعل هذا الكود؟ يعرض هذا الحل نموذج تسجيل طالب جديد فيه خانات للاسم والبريد والهاتف والعمر، واختيار الجنس والدورة، ومربع ملاحظات، وزر تسجيل.

`<h2>تسجيل طالب جديد</h2>`	عنوان فرعي h2 يظهر فوق النموذج.
`<form>`	الوسم form يبدأ النموذج الذي يضم كل خانات الإدخال.
`<p><label>الاسم الكامل: <input type="text" required></label></p>`	وضعنا الخانة input داخل الوسم label، فإذا ضغط المستخدم على النص «الاسم الكامل» ينتقل المؤشر إلى الخانة. الخاصية required تجعل الخانة إجبارية، فلا يرسل المتصفح النموذج إذا كانت فارغة. والوسم p يضع كل خانة في سطر خاص.
`<p><label>البريد: <input type="email"></label></p>`	خانة من نوع email يتحقق المتصفح عند الإرسال أن ما كُتب فيها يشبه بريدًا إلكترونيًا.
`<p><label>الهاتف: <input type="tel"></label></p>`	خانة من نوع tel مخصصة لرقم الهاتف، وعلى الجوال تظهر لوحة أرقام لتسهيل الكتابة.
`<p><label>العمر: <input type="number" min="15"></label></p>`	خانة أرقام number، والخاصية "min="15 تجعل أقل عمر مقبول هو 15.

`<p>الجنس:`	فقرة تبدأ بالنص «الجنس:» وتضم خياري الجنس في السطرين التاليين.
`<label><input type="radio" name="g"> ذكر</label>`	دائرة اختيار radio داخل label، فالضغط على كلمة «ذكر» يختار الدائرة. هذا الخيار والخيار التالي «أنثى» لهما نفس "name="g فلا يمكن اختيار الاثنين معًا.
`<p><label>الدورة:`	فقرة فيها label بعنوان «الدورة:» تحيط بالقائمة المنسدلة في السطر التالي.
`<select><option>HTML</option> <option>C#</option> <option>SQL</option></select> </label></p>`	الوسم select يصنع قائمة منسدلة، وبداخلها ثلاثة خيارات option: HTML وC# وSQL. ثم نغلق label والفقرة p.
`<p><label>ملاحظات: br><textarea rows="3" cols="30"></textarea> </label></p>`	الوسم textarea يصنع مربع نص كبير يسمح بكتابة عدة أسطر. الخاصية "rows="3 تجعل ارتفاعه 3 أسطر، و"cols="30 تجعل عرضه يتسع لحوالي 30 حرفًا، والوسم br يضع المربع تحت كلمة «ملاحظات».
`<button type="submit">تسجيل</button>`	زر مكتوب عليه «تسجيل»، و"type="submit تعني أنه يرسل النموذج عند الضغط عليه بعد التحقق من الخانات الإجبارية.
`</form>`	يغلق النموذج.

اختبر نفسك (3 أسئلة)

1. أي وسم يُستخدم لعنصر داخل قائمة؟

li | td | ul

2. خلية العنوان في الجدول هي:

td | th | tr

3. لاختيار خيار واحد فقط من عدة خيارات نستخدم:

checkbox | radio | textarea

الإجابات: 1) radio · 2) th · 3) li

الواجب: أضف إلى صفحتك جدولًا لجدولك الدراسي الأسبوعي.

تنسيق الصفحات بـ CSS

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يربط ملف CSS بصفحة HTML.
- يستخدم المحددات: الوسم والكلاس class والمعرّف id.
- يغير الألوان والخطوط والخلفيات.
- يفهم نموذج الصندوق: margin وborder وpadding.
- يرتب العناصر أفقيًا باستخدام Flexbox.

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. ما هو CSS وكيف نربطه؟

HTML يحدد **المحتوى**، وCSS يحدد **الشكل**. نكتب CSS داخل وسم `style` أو في ملف مستقل `style.css` نربطه بالصفحة.

```HTML
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

ماذا يفعل هذا الكود؟ يوضح هذا المثال كيف نربط صفحة HTML بملف تنسيق خارجي اسمه style.css، فتأخذ الصفحة ألوانها وشكلها من هذا الملف. لا يظهر شيء جديد بنفسه، لكن التنسيقات المكتوبة في الملف تُطبَّق على الصفحة.

`<head>` قسم head هو المكان المناسب لربط ملفات التنسيق، لأنه يحتوي معلومات الصفحة التي يقرؤها المتصفح قبل عرض المحتوى.

`<link rel="stylesheet" href="style.css">`	الوسم link يربط الصفحة بملف آخر. الخاصية rel="stylesheet" تقول إن هذا الملف ملف تنسيق CSS، والخاصية href تحدد اسم الملف style.css، ويجب أن يكون في نفس مجلد صفحة HTML في هذا المثال.
`</head>`	يغلق قسم head.

2. المحددات والخصائص

القاعدة تتكون من **محدد** ثم خصائص بين أقواس. الكلاس يبدأ بنقطة `.` ويمكن تكراره، والمعرّف يبدأ بـ `#` ويُستخدم مرة واحدة.

HTML

```
<style>
  body { font-family: Tahoma; background: #f5f5f5; }
  h1 { color: darkblue; text-align: center; }
  .highlight { background: yellow; }
  #intro { font-size: 20px; color: #444; }
</style>

<h1>مرحبًا بكم</h1>
<p id="intro">هذه فقرة تعريفية.</p>
<p>هذه فقرة فيها <span class="highlight">كلمة مميزة</span>.</p>
```

ماذا يفعل هذا الكود؟ يكتب هذا المثال تنسيق CSS داخل الصفحة نفسها: خلفية رمادية فاتحة وخط Tahoma للصفحة، وعنوان أزرق داكن في الوسط، وفقرة تعريفية أكبر بلون رمادي، وكلمة مميزة بخلفية صفراء.

`<style>`	الوسم style يسمح بكتابة تنسيقات CSS داخل ملف HTML نفسه. كل ما بين <style> و</style> قواعد تحدد شكل العناصر، ولا يظهر كنص على الشاشة.
`body { font-family: Tahoma; background: #f5f5f5; }`	هذه قاعدة CSS: كلمة body هي «المحدِّد» أي العنصر الذي نريد تنسيقه، وبين القوسين { } خصائص. font-family تجعل خط الصفحة Tahoma، وbackground تجعل الخلفية بلون #f5f5f5 وهو رمادي فاتح جدًا مكتوب بنظام الألوان السداسي (hex).
`h1 { color: darkblue; text-align: center; }`	تنسيق كل عناوين h1: الخاصية color تجعل لون النص أزرق داكنًا (darkblue)، والخاصية text-align: center تضع النص في منتصف السطر. لاحظ أن كل خاصية تنتهي بفاصلة منقوطة ;.
`.highlight { background: yellow; }`	النقطة قبل الاسم تعني أن هذا محدِّد «كلاس» (class)، فيُطبَّق التنسيق على أي عنصر مكتوب فيه class="highlight". هنا يجعل خلفيته صفراء مثل قلم التظليل.

`#intro { font-size: 20px; color: #444; }`	علامة # تعني محدِّد «معرّف» (id)، فيُطبَّق على العنصر الذي يحمل id="intro"، والمعرّف يُستخدم لعنصر واحد فقط في الصفحة. font-size: 20px تجعل حجم الخط 20 بكسل، وcolor: #444 لون رمادي غامق.
`</style>`	يغلق وسم التنسيق، وبعده يبدأ محتوى الصفحة الذي سيتأثر بهذه القواعد.
`<h1>مرحبًا بكم</h1>`	عنوان h1 سيظهر بلون أزرق داكن وفي منتصف الصفحة بسبب قاعدة h1 أعلاه.
`<p id="intro">هذه فقرة تعريفية.</p>`	فقرة تحمل المعرّف id="intro"، لذلك ستأخذ تنسيق #intro فتظهر بخط أكبر (20 بكسل) ولون رمادي غامق.
`<p>هذه فقرة فيها <span class="highlight">كلمة مميزة</span>. </p>`	الوسم span حاوية صغيرة تحيط بجزء من النص دون أن تنقله إلى سطر جديد. لأنه يحمل class="highlight" ستظهر عبارة «كلمة مميزة» بخلفية صفراء وباقي الفقرة عادي.

3. نموذج الصندوق (Box Model)

كل عنصر هو صندوق: **المحتوى** ثم `padding` (المسافة الداخلية) ثم `border` (الإطار) ثم `margin` (المسافة الخارجية).

HTML

```
<style>
  .card {
    background: white;
    padding: 20px;
    border: 2px solid #3b82f6;
    border-radius: 12px;
    margin: 20px;
    width: 250px;
  }
</style>
<div class="card">
  <h3>سارة أحمد</h3>
  <p>متدربة في دورة البرمجة</p>
</div>
```

ماذا يفعل هذا الكود؟ يصنع هذا المثال «بطاقة» بخلفية بيضاء وإطار أزرق وزوايا دائرية، وبداخلها اسم المتدربة ووصف قصير.

`<style>`	بداية قسم تنسيقات CSS داخل الصفحة.
`.card {`	بداية قاعدة للكلاس card؛ كل عنصر يحمل class="card" سيأخذ الخصائص المكتوبة في الأسطر التالية. هنا كتبنا كل خاصية في سطر لتسهيل القراءة.

`background: white;`	الخاصية background تجعل خلفية البطاقة بيضاء.
`padding: 20px;`	الخاصية padding هي المسافة الداخلية بين حدود البطاقة ومحتواها، وهنا 20 بكسل من كل الجهات حتى لا يلتصق النص بالإطار.
`border: 2px solid #3b82f6;`	الخاصية border ترسم إطارًا حول البطاقة: سُمكه 2px، ونوعه solid أي خط متصل، ولونه #3b82f6 وهو أزرق.
`border-radius: 12px;`	الخاصية border-radius تجعل زوايا البطاقة دائرية بدل أن تكون حادة، وكلما زاد الرقم زاد انحناء الزوايا.
`margin: 20px;`	الخاصية margin هي المسافة الخارجية حول البطاقة، أي الفراغ بينها وبين ما حولها، وهنا 20 بكسل.
`width: 250px;`	الخاصية width تحدد عرض البطاقة بـ 250 بكسل، بدل أن تمتد على عرض الصفحة كلها.
`}`	القوس } يغلق قاعدة .card، فكل الخصائص بين القوسين تابعة لها.
`<div class="card">`	الوسم div حاوية عامة نجمع بداخلها عناصر أخرى. لأنه يحمل "class="card سيظهر بشكل البطاقة الموصوف في التنسيق.
`<h3>سارة أحمد</h3>`	عنوان h3 داخل البطاقة يعرض اسم المتدربة.
`<p>متدربة في دورة البرمجة</p>`	فقرة داخل البطاقة تعرض وصفًا قصيرًا تحت الاسم.
`</div>`	يغلق الحاوية div، فالعنوان والفقرة كلاهما داخل البطاقة.

4. ترتيب العناصر بـ Flexbox

`display: flex` يضع العناصر الأبناء بجانب بعضها. `gap` يضع مسافة بينها، و `justify-content` يحدد توزيعها.

HTML

```
<style>
  nav { display: flex; gap: 16px; background: #1e293b; padding: 12px; }
  nav a { color: white; text-decoration: none; }
  nav a:hover { color: #fbbf24; }
</style>
<nav>
  <a href="#">الرئيسية</a>
  <a href="#">الطلاب</a>
  <a href="#">التسجيل</a>
</nav>
```

ماذا يفعل هذا الكود؟ يصنع هذا المثال شريط تنقل داكنًا فيه ثلاثة روابط بيضاء متجاورة في صف واحد، ويتغير لون الرابط إلى الأصفر عند مرور الفأرة فوقه.

`<style>`	بداية قسم تنسيقات CSS.
`nav { display: flex; gap: 16px; background: #1e293b; padding: 12px; }`	تنسيق شريط nav: الخاصية display: flex ترتّب العناصر التي بداخله بجانب بعضها في صف واحد، وgap: 16px تضع مسافة 16 بكسل بين كل عنصر والذي يليه. background تعطيه لونًا كحليًا داكنًا، وpadding مسافة داخلية 12 بكسل.
`nav a { color: white; text-decoration: none; }`	المحدِّد nav a يعني «كل رابط a موجود داخل nav» فقط. color: white يجعل لون الروابط أبيض، وtext-decoration: none يزيل الخط الذي يظهر تحت الروابط عادةً.
`nav a:hover { color: #fbbf24; }`	الإضافة :hover تعني «عندما تمر الفأرة فوق العنصر». فعندما تضع الفأرة على أي رابط يتغير لونه إلى #fbbf24 وهو أصفر ذهبي، ويعود أبيض عند إبعادها.
`</style>`	يغلق قسم التنسيقات.
`<nav>`	الوسم nav يمثل شريط التنقل، وسيأخذ الخلفية الداكنة وترتيب flex من التنسيق أعلاه.
`<a href="#">الرئيسية</a>`	رابط a مؤقت (href="#") لا ينقل لصفحة أخرى). هذا الرابط والرابطان بعده يظهرون بجانب بعضهم بلون أبيض بفضل display: flex.
`</nav>`	يغلق شريط التنقل.

:hover تنسيق يظهر عند مرور الفأرة فوق العنصر.

التمارين العملية

تمرين 1: نسّق صفحة التعريف

سهل

أضف CSS لصفحة يوم 1: لون خلفية فاتح، عنوان رئيسي بلون مختلف ومتوسط، خط Tahoma، وصورة بحواف دائرية (border-radius).

الحل المقترح:

HTML

```
<style>
  body { background: #eef2f7; font-family: Tahoma; }
  h1 { color: #0f766e; text-align: center; }
  img { border-radius: 50%; display: block; margin: auto; }
```

```
</style>
<h1>أحمد علي</h1>
<img src="https://placehold.co/150" alt="صورتي">
<p>أنا طالب أحب البرمجة.</p>
```

ماذا يفعل هذا الكود؟ يعرض هذا الحل صفحة تعريف بخلفية زرقاء فاتحة، فيها اسم في الوسط بلون أخضر مزرق، وتحته صورة دائرية في منتصف الصفحة، ثم جملة تعريفية.

`<style>`	بداية قسم تنسيقات CSS.
`body { background: #eef2f7; font-family: Tahoma; }`	تنسيق الصفحة كلها: خلفية بلون #eef2f7 (أزرق رمادي فاتح جدًا)، والخط Tahoma.
`h1 { color: #0f766e; text-align: center; }`	العنوان h1 يظهر بلون #0f766e (أخضر مزرق) وفي منتصف السطر بفضل text-align: center.
`img { border-radius: 50%; display: block; margin: auto; }`	تنسيق الصورة: border-radius: 50% يجعل الصورة المربعة دائرة كاملة. display: block يجعل الصورة تأخذ سطرًا خاصًا بها، وmargin: auto يوزع المسافة الفارغة على الجانبين بالتساوي فتصبح الصورة في منتصف الصفحة أفقيًا.
`</style>`	يغلق قسم التنسيقات.
`<h1>أحمد علي</h1>`	عنوان h1 يعرض الاسم بالتنسيق المحدد أعلاه.
`<img src="https://placehold.co/150" alt="صورتي">`	صورة تجريبية مربعة بحجم 150 بكسل، وستظهر دائرية وفي المنتصف بسبب قاعدة img.
`<p>أنا طالب أحب البرمجة.</p>`	فقرة تعريفية تظهر تحت الصورة.

تمرين 2: بطاقات الطلاب

متوسط

أنشئ 3 بطاقات طلاب (اسم + دورة + درجة) متجاورة باستخدام flex، لكل بطاقة إطار وحواف دائرية ومسافة داخلية.

الحل المقترح:

HTML

```
<style>
  .cards { display: flex; gap: 16px; flex-wrap: wrap; }
  .card { border: 1px solid #ccc; border-radius: 10px; padding: 16px; width: 180px;
background: white; }
  .card h3 { margin: 0; color: #6d3fc0; }
</style>
<div class="cards">
```

```
  <div class="card"><h3>سارة</h3><p>C#</p><b>95</b></div>
  <div class="card"><h3>خالد</h3><p>HTML</p><b>88</b></div>
  <div class="card"><h3>مريم</h3><p>SQL</p><b>91</b></div>
</div>
```

ماذا يفعل هذا الكود؟ يعرض هذا الحل ثلاث بطاقات طلاب متجاورة في صف، كل بطاقة فيها اسم الطالب ولغته ودرجته، وتنتقل البطاقات إلى سطر جديد إذا ضاقت الشاشة.

`<style>`	بداية قسم تنسيقات CSS.
`.cards { display: flex; gap: 16px; flex-wrap: wrap; }`	تنسيق الحاوية التي تحمل الكلاس cards: display: flex يضع البطاقات بجانب بعضها، وgap: 16px يترك مسافة بينها، وflex-wrap: wrap يسمح للبطاقات بالنزول إلى سطر جديد إذا لم يكفِ عرض الشاشة.
`.card { border: 1px solid #ccc; border-radius: 10px; padding: 16px; width: 180px; background: white; }`	تنسيق كل بطاقة: إطار رمادي فاتح رفيع (1px)، وزوايا دائرية 10 بكسل، ومسافة داخلية 16 بكسل، وعرض ثابت 180 بكسل، وخلفية بيضاء.
`.card h3 { margin: 0; color: #6d3fc0; }`	المحدِّد .card h3 يعني «كل عنوان h3 داخل عنصر له الكلاس card.». margin: 0 يلغي المسافة الخارجية التي يضعها المتصفح للعناوين تلقائيًا، وcolor يجعل لونه بنفسجيًا.
`</style>`	يغلق قسم التنسيقات.
`<div class="cards">`	حاوية div بالكلاس cards تجمع البطاقات الثلاث وترتبها في صف بفضل flex.
`<div class="card"><h3>سارة</h3> <p>C#</p><b>95</b></div>`	بطاقة واحدة: div بالكلاس card بداخلها عنوان h3 بالاسم، وفقرة p باللغة، والوسم b يعرض الدرجة 95 بخط عريض. السطران التاليان بطاقتان أخريان بنفس الشكل لخالد ومريم.
`</div>`	يغلق الحاوية الخارجية cards.

تمرين 3: تحدٍّ: الصفحة الرئيسية للنظام

تحدٍّ

اجمع ما تعلمته: header فيه اسم النظام، nav بثلاثة روابط، main فيه فقرة ترحيب ونموذج التسجيل من يوم 2، وfooter. نسّقها بشكل مرتب. هذا هو الجزء الأول من مشروعك النهائي.

الحل المقترح:

HTML

```
<style>
  body { margin: 0; font-family: Tahoma; background: #f4f6f8; }
```

```
  header { background: #0f766e; color: white; padding: 16px; }
  nav { display: flex; gap: 16px; background: #134e4a; padding: 10px 16px; }
  nav a { color: white; text-decoration: none; }
  main { padding: 20px; max-width: 600px; margin: auto; }
  form { background: white; padding: 16px; border-radius: 10px; }
  input, select { padding: 6px; width: 100%; margin-bottom: 10px; }
  button { background: #0f766e; color: white; border: 0; padding: 8px 20px; border-
radius: 6px; }
  footer { text-align: center; color: #666; padding: 16px; }
</style>
<header><h1>نظام سجل الطلاب</h1></header>
<nav><a href="#">الرئيسية</a><a href="#">الطلاب</a><a href="#">تسجيل</a></nav>
<main>
  <p>مرحبًا بك في نظام تسجيل الطلاب.</p>
  <form>
    <label>الاسم</label><input type="text">
    <label>البريد</label><input type="email">
    <label>الدورة</label><select><option>C#</option><option>HTML</option></select>
    <button>تسجيل</button>
  </form>
</main>
<footer>© 2026</footer>
```

ماذا يفعل هذا الكود؟ يعرض هذا الحل صفحة كاملة منسقة لنظام سجل الطلاب: رأس ملوّن بالعنوان، وشريط روابط، ومحتوى في الوسط فيه نموذج تسجيل أبيض، وتذييل بحقوق النشر في الأسفل.

`<style>`	بداية قسم تنسيقات CSS للصفحة كلها.
`body { margin: 0; font-family: Tahoma; background: #f4f6f8; }`	margin: 0 يزيل المسافة الصغيرة التي يضعها المتصفح حول الصفحة تلقائيًا، فيلتصق الرأس بحواف النافذة. ونحدد الخط Tahoma وخلفية رمادية فاتحة.
`header { background: #0f766e; color: white; padding: 16px; }`	رأس الصفحة header يأخذ خلفية خضراء مزرقة ونصًا أبيض ومسافة داخلية 16 بكسل.
`nav { display: flex; gap: 16px; background: #134e4a; padding: 10px 16px; }`	شريط التنقل يرتب روابطه في صف بـ display: flex مع مسافة بينها. عندما نكتب قيمتين في padding مثل 10px 16px فالأولى للأعلى والأسفل والثانية لليمين واليسار.
`nav a { color: white; text-decoration: none; }`	روابط الشريط بيضاء وبدون خط تحتها.
`main { padding: 20px; max-width: 600px; margin: auto; }`	المحتوى الرئيسي main: الخاصية max-width: 600px تمنع عرضه من تجاوز 600 بكسل، وmargin: auto تضعه في منتصف الصفحة أفقيًا، وpadding يترك مسافة داخلية.

النموذج form يظهر كصندوق أبيض بمسافة داخلية وزوايا دائرية.	`form { background: white; padding: 16px; border-radius: 10px; }`
الفاصلة بين input وselect تعني تطبيق نفس التنسيق على النوعين معًا. width: 100% يجعل كل خانة بعرض النموذج كاملًا، و:margin-bottom 10px يترك مسافة تحت كل خانة.	`input, select { padding: 6px; width: 100%; margin-bottom: 10px; }`
تنسيق الزر: خلفية خضراء مزرقة ونص أبيض، و0 :border يزيل الإطار الافتراضي للزر، مع مسافة داخلية وزوايا دائرية ليبدو أجمل.	`button { background: #0f766e; color: white; border: 0; padding: 8px 20px; border-radius: 6px; }`
التذييل footer نصه في المنتصف وبلون رمادي مع مسافة داخلية.	`footer { text-align: center; color: #666; padding: 16px; }`
يغلق قسم التنسيقات، وبعده يبدأ محتوى الصفحة.	`</style>`
رأس الصفحة وبداخله العنوان الرئيسي، وسيظهر بشريط أخضر عريض في أعلى الصفحة.	`<header><h1>نظام سجل الطلاب</h1></header>`
شريط التنقل بثلاثة روابط مؤقتة. لم نكتب فواصل بينها لأن gap في CSS يضع المسافة بينها تلقائيًا.	`<nav><a href="#">الرئيسية</a><a href="#">الطلاب</a><a href="#">تسجيل</a></nav>`
بداية المحتوى الرئيسي الذي سيظهر في منتصف الصفحة.	`<main>`
فقرة ترحيبية فوق النموذج.	`<p>مرحبًا بك في نظام تسجيل الطلاب.</p>`
بداية نموذج التسجيل الذي يظهر كصندوق أبيض.	`<form>`
نص وصفي «الاسم» ثم خانة نصية. لأن عرض الخانة 100% ستنزل تحت النص وتملأ عرض النموذج. والسطر التالي يفعل نفس الشيء لخانة البريد من نوع email.	`<label>الاسم</label><input type="text">`
نص وصفي ثم قائمة منسدلة select فيها خياران: C# وHTML، وتأخذ نفس تنسيق الخانات.	`<label>الدورة</label><select><option>C#</option><option>HTML</option></select>`
زر «تسجيل» بالتنسيق الأخضر. الزر داخل form بدون type يكون من نوع submit تلقائيًا، أي يرسل النموذج عند الضغط.	`<button>تسجيل</button>`
يغلق النموذج، والسطر التالي يغلق المحتوى الرئيسي main.	`</form>`
تذييل الصفحة يعرض رمز حقوق النشر © والسنة في منتصف أسفل الصفحة.	`<footer>© 2026</footer>`

اختبر نفسك (3 أسئلة)

1. أي محدد يختار العناصر التي لها "class="box؟

box | box. | box#

2. المسافة بين محتوى العنصر وإطاره هي:

gap | padding | margin

3. لوضع العناصر بجانب بعضها نستخدم:

display: flex | color: flex | text-align: flex

الإجابات: 1) box. · 2) padding · 3) display: flex.

الواجب: أكمل الصفحة الرئيسية لمشروعك (تحدي اليوم) وأحضرها يوم 5.

C# اليوم 4 · الأسبوع 1 · ساعتان · دورة أساسيات البرمجة

أول برنامج بـ C#: المتغيرات وأنواع البيانات

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- ينشئ مشروع Console App في Visual Studio ويشغله.
- يطبع النصوص بـ Console.WriteLine.
- يعرّف متغيرات من الأنواع int وdouble وstring وbool.
- يقرأ مدخلات المستخدم ويحولها إلى أرقام.
- يدمج المتغيرات في النصوص باستخدام $"...".

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. إنشاء أول مشروع

افتح **Visual Studio Community** (C#) ← اختر أحدث إصدار ← Create a new project ← **Console App** يظهر في القائمة ← Create. ثم اضغط `F5` أو زر التشغيل الأخضر.

C#

```
// هذا تعليق: لا ينفذه الحاسوب
Console.WriteLine("مرحبًا بالعالم!");
Console.WriteLine("أنا أتعلم C#");
```

```
Output ▸ مرحبًا بالعالم!
أنا أتعلم C#
```

ماذا يفعل هذا الكود؟ برنامج بسيط يطبع سطرين على الشاشة: «مرحبًا بالعالم!» ثم «أنا أتعلم C#». السطر الأول تعليق لا يظهر عند التشغيل.

`// هذا تعليق: لا ينفذه الحاسوب`	الرمزان // يبدآن «تعليقًا»، وهو ملاحظة نكتبها للبشر داخل الكود. الحاسوب يتجاهل كل ما بعد // حتى نهاية السطر، فلا يؤثر على عمل البرنامج.
`Console.WriteLine("مرحبًا بالعالم!");`	Console هي «الشاشة السوداء» التي يكتب فيها البرنامج، و WriteLine أمر يطبع النص الموجود بين القوسين ثم ينتقل إلى سطر جديد. النص يوضع بين علامتي تنصيص "..."، والفاصلة المنقوطة ; في النهاية تعني «انتهت هذه التعليمة» ويجب وضعها بعد كل تعليمة في C#.
`Console.WriteLine("أنا أتعلم C#");`	تعليمة ثانية من نفس النوع تطبع «أنا أتعلم C#» في سطر جديد. الحاسوب ينفذ التعليمات بالترتيب من الأعلى إلى الأسفل، لذلك يظهر هذا السطر بعد الأول.

كل أمر في C# ينتهي بفاصلة منقوطة ; ونسيانها هو أكثر خطأ شائع عند المبتدئين. ولغة C# حساسة لحالة الأحرف: Console غير console.

2. المتغيرات وأنواع البيانات

المتغير صندوق له **نوع** و**اسم** و**قيمة**.

النوع	يخزن	مثال
`int`	عدد صحيح	`int age = 20;`
`double`	عدد عشري	`double price = 9.5;`
`string`	نص	`string name = "Ali";`
`bool`	صح/خطأ	`bool passed = true;`
`char`	حرف واحد	`char grade = 'A';`

C#

```
string name = "سارة";
int age = 22;
double gpa = 3.75;
bool isStudent = true;

Console.WriteLine($"الاسم: {name}");
Console.WriteLine($"العمر: {age} - المعدل: {gpa}");
Console.WriteLine($"طالبة؟ {isStudent}");
```

```
Output ▸ الاسم: سارة
العمر: 22 - المعدل: 3.75
```

```
True طالبة؟
```

ماذا يفعل هذا الكود؟ ينشئ البرنامج أربعة متغيرات من أنواع مختلفة (نص، عدد صحيح، عدد عشري، صح/خطأ) ثم يطبع قيمها داخل جمل. سيرى الطالب: الاسم: سارة، ثم العمر والمعدل، ثم «طالبة؟ True».

`string name = "سارة";`	هنا ننشئ «متغيرًا»، وهو صندوق في الذاكرة له اسم ويحفظ قيمة. الكلمة string تعني أن نوعه «نص»، واسمه name، والعلامة = تعني «ضع القيمة التي على اليمين داخل المتغير»، فيصبح name يحتوي على النص "سارة".
`int age = 22;`	الكلمة int تعني «عدد صحيح» أي عدد بدون كسور. أنشأنا متغيرًا اسمه age ووضعنا فيه 22، ولاحظ أن الأعداد لا توضع بين علامات تنصيص.
`double gpa = 3.75;`	الكلمة double تعني «عدد عشري» يمكن أن يحتوي على فاصلة عشرية. المتغير gpa يحفظ المعدل 3.75، ونكتب الفاصلة العشرية في الكود بنقطة دائمًا.
`bool isStudent = true;`	الكلمة bool نوع لا يحمل إلا قيمتين: true (صحيح) أو false (خطأ). المتغير isStudent يحفظ true أي «نعم، هي طالبة».
`Console.WriteLine($"الاسم: {name}");`	العلامة $ قبل النص تسمح لنا بوضع متغيرات داخل النص بين قوسين معقوفين { }، وهذا يسمى «إدراج القيم في النص». عند التشغيل يُستبدل {name} بقيمته فيظهر: الاسم: سارة.
`Console.WriteLine($"العمر: {age} - المعدل: {gpa}");`	يمكن إدراج أكثر من متغير في النص نفسه. هنا يُستبدل {age} بـ 22 و {gpa} بـ 3.75، فيظهر العمر والمعدل في سطر واحد.
`Console.WriteLine($"طالبة؟ {isStudent}");`	عند طباعة متغير من نوع bool تكتب C# قيمته بالإنجليزية وبحرف كبير في أوله، فيظهر على الشاشة: طالبة؟ True.

3. قراءة مدخلات المستخدم

`Console.ReadLine()` يقرأ ما يكتبه المستخدم كنص دائمًا. لتحويله إلى رقم نستخدم `int.Parse` أو `double.Parse`.

C#

```
Console.Write("ما اسمك؟ ");
string name = Console.ReadLine();

Console.Write("كم عمرك؟ ");
int age = int.Parse(Console.ReadLine());

Console.WriteLine($"أهلًا {name}، بعد 5 سنوات سيكون عمرك {age + 5}");
```

```
Output ▸ ما اسمك؟ خالد
كم عمرك؟ 20
أهلًا خالد، بعد 5 سنوات سيكون عمرك 25
```

ماذا يفعل هذا الكود؟ يسأل البرنامج المستخدم عن اسمه وعمره، ثم يرحب به ويخبره كم سيكون عمره بعد 5 سنوات. مثلًا إذا كتب «أحمد» و 20 يظهر: أهلًا أحمد، بعد 5 سنوات سيكون عمرك 25.

`Console.Write("ما اسمك؟ ");`	الأمر Write يطبع النص مثل WriteLine لكنه لا ينتقل إلى سطر جديد، فيبقى المؤشر بجانب السؤال ليكتب المستخدم إجابته في نفس السطر.
`string name = Console.ReadLine();`	الأمر Console.ReadLine ينتظر حتى يكتب المستخدم شيئًا ويضغط Enter، ثم يعيد ما كتبه على شكل نص (string). نحفظ هذا النص في المتغير name لنستخدمه لاحقًا.
`Console.Write("كم عمرك؟ ");`	نطبع السؤال الثاني بنفس الطريقة، دون الانتقال إلى سطر جديد.
`int age = int.Parse(Console.ReadLine());`	ReadLine يعيد دائمًا نصًا حتى لو كتب المستخدم أرقامًا، والنص "20" لا يمكن جمعه كعدد. لذلك نستخدم int.Parse الذي يحوّل النص إلى عدد صحيح int، وإذا كتب المستخدم شيئًا ليس عددًا يتوقف البرنامج برسالة خطأ.
`Console.WriteLine($"أهلًا {name}، بعد 5 سنوات سيكون عمرك {age + 5}");`	داخل الأقواس { } في النص الذي يبدأ بـ $ يمكن وضع عملية حسابية وليس متغيرًا فقط. هنا {age + 5} يُحسب أولًا ثم تُطبع النتيجة، أما المتغير age نفسه فلا تتغير قيمته.

لعرض العربية بشكل صحيح في الكونسول أضف في أول البرنامج: `Console.OutputEncoding = System.Text.Encoding.UTF8;`

التمارين العملية

تمرين 1: برنامج ترحيب

سهل

اطلب من المستخدم اسمه ومدينته، ثم اطبع: «مرحبًا [الاسم] من [المدينة]!».

الحل المقترح:

C#

```
Console.Write("اسمك: ");
string name = Console.ReadLine();
Console.Write("مدينتك: ");
string city = Console.ReadLine();
Console.WriteLine($"مرحبًا {name} من {city}!");
```

ماذا يفعل هذا الكود؟ يسأل البرنامج المستخدم عن اسمه ومدينته ثم يطبع ترحيبًا يجمعهما، مثل: مرحبًا سارة من مسقط!

`Console.Write("اسمك: ");`	Console.Write يطبع السؤال ويترك المؤشر في نفس السطر حتى يكتب المستخدم بجانبه.
`string name = Console.ReadLine();`	Console.ReadLine يقرأ ما يكتبه المستخدم حتى يضغط Enter ويعيده كنص، فنحفظه في المتغير name من نوع string.
`Console.Write("مدينتك: ");`	نطبع السؤال الثاني عن المدينة بنفس الطريقة.
`string city = Console.ReadLine();`	نقرأ اسم المدينة ونحفظه في متغير نصي جديد اسمه city. لا نحتاج إلى تحويل هنا لأننا نريد نصًا أصلًا.
`Console.WriteLine($"مرحبًا {name} من {city}!");`	النص الذي يبدأ بـ $ يسمح بإدراج المتغيرات بين { }، فيُستبدل {name} و {city} بما كتبه المستخدم ثم يُطبع السطر كاملًا.

تمرين 2: مساحة المستطيل

متوسط

اقرأ الطول والعرض (أعداد عشرية) واطبع المساحة والمحيط.

الحل المقترح:

C#

```
Console.Write("الطول: ");
double length = double.Parse(Console.ReadLine());
Console.Write("العرض: ");
double width = double.Parse(Console.ReadLine());

double area = length * width;
double perimeter = 2 * (length + width);
Console.WriteLine($"المساحة = {area}");
Console.WriteLine($"المحيط = {perimeter}");
```

ماذا يفعل هذا الكود؟ يطلب البرنامج طول المستطيل وعرضه، ثم يحسب مساحته ومحيطه ويطبعهما. مثلًا مع طول 5 وعرض 3 يظهر: المساحة = 15 ثم المحيط = 16.

`Console.Write("الطول: ");`	نطبع طلبًا للمستخدم ليكتب الطول، ويبقى المؤشر في نفس السطر لأننا استخدمنا Write وليس WriteLine.
`double length = double.Parse(Console.ReadLine());`	ReadLine يعيد نصًا، و double.Parse يحوّله إلى عدد عشري double حتى نستطيع إجراء الحساب عليه، ويقبل أعدادًا بكسور مثل 2.5. نحفظ النتيجة في المتغير length.
`Console.Write("العرض: ");`	نطلب من المستخدم إدخال العرض.

نقرأ العرض ونحوّله إلى عدد عشري بنفس الطريقة ونحفظه في width.	`double width = double.Parse(Console.ReadLine());`
العلامة * تعني الضرب في C#. نضرب الطول في العرض للحصول على المساحة ونحفظ الناتج في متغير جديد اسمه area.	`double area = length * width;`
الأقواس ( ) تجعل الجمع يُنفذ أولًا، ثم يُضرب الناتج في 2. بدون الأقواس كانت C# ستضرب 2 في length أولًا لأن الضرب يسبق الجمع، فتكون النتيجة خاطئة.	`double perimeter = 2 * (length + width);`
نطبع المساحة بإدراج المتغير area داخل النص باستخدام $ والأقواس { }.	`Console.WriteLine($"المساحة = {area}");`
نطبع المحيط في سطر جديد بنفس الطريقة.	`Console.WriteLine($"المحيط = {perimeter}");`

تمرين 3: تحدٍّ: تحويل الحرارة

تحدٍّ

اقرأ درجة الحرارة بالمئوية وحوّلها إلى فهرنهايت بالمعادلة F = C × 9 / 5 + 32.

الحل المقترح:

C#

```
Console.Write("الحرارة بالمئوية: ");
double c = double.Parse(Console.ReadLine());
double f = c * 9 / 5 + 32;
Console.WriteLine($"{c}°C = {f}°F");
```

```
الحرارة بالمئوية: Output ▸ 30
30°C = 86°F
```

ماذا يفعل هذا الكود؟ يطلب البرنامج درجة الحرارة بالمئوية ويحوّلها إلى فهرنهايت. مثلًا إذا أدخل المستخدم 100 يظهر: 100°C = 212°F.

نطبع طلبًا للمستخدم ليكتب درجة الحرارة، ويبقى المؤشر بجانبه لأننا استخدمنا Write.	`Console.Write("الحرارة بالمئوية: ");`
نقرأ ما كتبه المستخدم كنص بـ ReadLine ثم نحوّله إلى عدد عشري بـ double.Parse، ونحفظه في المتغير c.	`double c = double.Parse(Console.ReadLine());`

`double f = c * 9 / 5 + 32;`	هذه معادلة التحويل: C# تنفذ الضرب * والقسمة / أولًا من اليسار إلى اليمين، ثم الجمع + في الأخير. ولأن c من نوع double فالنتيجة تحتفظ بالكسور ولا تضيع.
`Console.WriteLine($"{c}°C = {f}°F");`	نطبع القيمتين داخل نص واحد باستخدام $ و { }، والرموز °C و °F مجرد نص عادي يُطبع كما هو.

اختبر نفسك (3 أسئلة)

1. أي نوع يناسب تخزين السعر 12.5؟

int · double · bool

2. ماذا يُرجع Console.ReadLine()؟

رقمًا · نصًا string · قيمة bool

3. ما الذي ينقص هذا السطر؟ int x = 5

فاصلة منقوطة ; · قوسان () · علامة $

الإجابات: 1) double · 2) نصًا string · 3) فاصلة منقوطة ;

الواجب: اكتب برنامجًا يقرأ اسم منتج وسعره وكميته ويطبع الإجمالي.

C# اليوم 5 · الأسبوع 1 · ساعتان · دورة أساسيات البرمجة

العمليات والشروط + اختبار منتصف الدورة

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يستخدم العمليات الحسابية + − * / % بشكل صحيح.
- يقارن القيم بـ == و!= و> و<.
- يجمع الشروط بـ && و|| و!.
- يكتب قرارات بـ if / else if / else.
- يستخدم switch لاختيار من عدة حالات.

توزيع وقت الحصة

0:00 – 0:10	مراجعة المتغيرات والإدخال
0:10 – 0:45	شرح العمليات الحسابية والمنطقية وif وswitch
0:45 – 1:20	تمارين عملية على الشروط
1:20 – 1:30	استراحة
1:30 – 1:55	اختبار منتصف الدورة (HTML + أساسيات C#)
1:55 – 2:00	ملاحظات ختام الأسبوع الأول

شرح الدرس

1. العمليات الحسابية

`%` يعطي **باقي القسمة**، ومفيد لمعرفة الزوجي والفردي. انتبه: قسمة عددين صحيحين تعطي عددًا صحيحًا: `7 / 2` = 3.

C#

```
int a = 17, b = 5;
Console.WriteLine(a + b);   // 22
Console.WriteLine(a - b);   // 12
Console.WriteLine(a * b);   // 85
Console.WriteLine(a / b);   // 3
Console.WriteLine(a % b);   // 2
Console.WriteLine(7.0 / 2); // 3.5
```

ماذا يفعل هذا الكود؟ يجرب البرنامج العمليات الحسابية الأساسية على العددين 17 و 5 ويطبع ناتج كل عملية في سطر. يرى الطالب: 22 ثم 12 ثم 85 ثم 3 ثم 2 ثم 3.5.

`int a = 17, b = 5;` نُنشئ متغيرين من نوع int (عدد صحيح) في سطر واحد، ونفصل بينهما بفاصلة: a قيمته 17 و b قيمته 5.

`Console.WriteLine(a + b);  // 22`	العلامة + تجمع العددين، و WriteLine يطبع الناتج 22. النص بعد // تعليق يوضح النتيجة المتوقعة ولا يُنفذ.
`Console.WriteLine(a - b);  // 12`	العلامة - للطرح، فيُطبع 12.
`Console.WriteLine(a * b);  // 85`	العلامة * للضرب في C# (وليس ×)، فيُطبع 85.
`Console.WriteLine(a / b);  // 3`	العلامة / للقسمة. لكن عندما يكون العددان صحيحين int تكون القسمة «صحيحة»: يُحذف الكسر وتبقى النتيجة 3 بدلًا من 3.4.
`Console.WriteLine(a % b);  // 2`	العلامة % تعطي «باقي القسمة»: 17 مقسومة على 5 تساوي 3 والباقي 2، فيُطبع 2. هذه العملية مفيدة جدًا، مثلًا لمعرفة هل العدد زوجي أم فردي.
`Console.WriteLine(7.0 / 2); // 3.5`	كتبنا 7.0 بفاصلة عشرية فأصبح عددًا عشريًا double، وعندما يكون أحد طرفي القسمة عشريًا تحتفظ النتيجة بالكسر فيُطبع 3.5.

2. المقارنة والمنطق

العملية	المعنى
`==`	يساوي
`!=`	لا يساوي
`> < >= <=`	أكبر، أصغر، أكبر أو يساوي، أصغر أو يساوي
`&&`	و (الشرطان صحيحان)
`||`	أو (أحدهما صحيح)
`!`	نفي

3. الجملة الشرطية if

ينفذ البرنامج الجزء الأول الذي يتحقق شرطه فقط، ويتجاهل الباقي.

```
C#

Console.Write("أدخل الدرجة: ");
int score = int.Parse(Console.ReadLine());

if (score >= 90)
    Console.WriteLine("ممتاز");
else if (score >= 80)
    Console.WriteLine("جيد جدًا");
```

```
else if (score ⩾ 60)
    Console.WriteLine("ناجح");
else
    Console.WriteLine("راسب");
```

```
Output ▸ 85 :أدخل الدرجة
جيد جدًا
```

ماذا يفعل هذا الكود؟ يطلب البرنامج درجة الطالب ثم يطبع تقديرًا واحدًا حسبها: ممتاز أو جيد جدًا أو ناجح أو راسب. مثلًا الدرجة 85 تطبع «جيد جدًا».

`Console.Write("أدخل الدرجة: ");`	نطبع طلبًا للمستخدم ليكتب الدرجة، ويبقى المؤشر في نفس السطر.
`int score = int.Parse(Console.ReadLine());`	ReadLine يعيد ما كتبه المستخدم كنص، و int.Parse يحوّله إلى عدد صحيح حتى نستطيع مقارنته بأعداد أخرى. نحفظه في score.
`if (score ⩾ 90)`	الكلمة if تعني «إذا»: تفحص الشرط بين القوسين، فإن كان صحيحًا تُنفذ التعليمة التي تحته. العلامة >= تعني «أكبر من أو يساوي».
`Console.WriteLine("ممتاز");`	هذا السطر يُنفذ فقط إذا كانت الدرجة 90 أو أكثر. عندما لا نستخدم الأقواس { } يتبع if سطر واحد فقط هو الذي بعده مباشرة.
`else if (score ⩾ 80)`	else if تعني «وإلا، إذا...»: لا يُفحص هذا الشرط إلا إذا كان الشرط السابق خاطئًا. أي أننا نصل هنا فقط إذا كانت الدرجة أقل من 90.
`Console.WriteLine("جيد جدًا");`	يُطبع «جيد جدًا» إذا كانت الدرجة من 80 إلى 89.
`else if (score ⩾ 60)`	شرط ثالث يُفحص فقط إذا فشل الشرطان السابقان، أي الدرجة أقل من 80.
`Console.WriteLine("ناجح");`	يُطبع «ناجح» إذا كانت الدرجة من 60 إلى 79.
`else`	else وحدها تعني «في كل الحالات الأخرى» ولا تحتاج شرطًا. تُنفذ فقط إذا كانت كل الشروط السابقة خاطئة، أي الدرجة أقل من 60.
`Console.WriteLine("راسب");`	يُطبع «راسب». لاحظ أن البرنامج ينفذ فرعًا واحدًا فقط من السلسلة: أول شرط صحيح ثم يتجاهل الباقي.

خطأ شائع: كتابة = (إسناد) بدل == (مقارنة) داخل الشرط.

4. الجملة switch

تناسب الاختيار من قيم محددة، مثل قوائم البرامج.

```
C#
```

```
Console.Write("اختر (1 إضافة، 2 عرض، 0 خروج): ");
string choice = Console.ReadLine();

switch (choice)
{
    case "1":
        Console.WriteLine("إضافة طالب.  ");
        break;
    case "2":
        Console.WriteLine("عرض الطلاب.  ");
        break;
    case "0":
        Console.WriteLine("وداعًا");
        break;
    default:
        Console.WriteLine("اختيار غير صحيح");
        break;
}
```

ماذا يفعل هذا الكود؟ يعرض البرنامج قائمة اختيارات ويقرأ اختيار المستخدم، ثم يطبع رسالة مناسبة: إضافة أو عرض أو وداع، أو «اختيار غير صحيح» إذا كتب شيئًا آخر.

`Console.Write("اختر (1 إضافة، 2 عرض، 0 خروج): ");`	نطبع قائمة الاختيارات المتاحة ليعرف المستخدم ماذا يكتب، ويبقى المؤشر في نفس السطر.
`string choice = Console.ReadLine();`	نقرأ اختيار المستخدم كنص ونحفظه في choice. لم نحوّله إلى عدد لأننا سنقارنه بنصوص مثل "1".
`switch (choice)`	الكلمة switch تعني «بدّل حسب»: تأخذ قيمة choice وتقارنها بعدة قيم محتملة، وتنفذ الجزء المطابق. هي بديل أوضح لسلسلة طويلة من if و else if.
`{`	الأقواس المعقوفة { } تجمع مجموعة من التعليمات في «كتلة» واحدة، وهنا تحدد بداية ونهاية جسم الـ switch.
`case "1":`	case تعني «حالة»: إذا كانت قيمة choice تساوي النص "1" يبدأ التنفيذ من هنا. لاحظ علامة النقطتين : في نهاية السطر.
`Console.WriteLine("إضافة طالب.  ");`	هذه التعليمة تُنفذ عندما يختار المستخدم 1.
`break;`	break تعني «اخرج من الـ switch الآن». في C# يجب أن ينتهي كل case بـ break (أو ما يشبهها) حتى لا ينتقل التنفيذ إلى الحالة التالية، وإلا يرفض المترجم البرنامج.
`case "2":`	الحالة الثانية: إذا كتب المستخدم "2" تُنفذ التعليمات التي تحتها.

`Console.WriteLine("عرض الطلاب. ");`	يُطبع «عرض الطلاب...» ثم تأتي break للخروج.
`case "0":`	الحالة الثالثة: إذا كتب المستخدم "0" للخروج.
`Console.WriteLine("وداعًا");`	يُطبع «وداعًا» ثم break تُنهي الـ switch.
`default:`	default تعني «الحالة الافتراضية»: تُنفذ إذا لم تطابق القيمة أي case من الحالات السابقة، مثل أن يكتب المستخدم 5 أو حرفًا.
`Console.WriteLine("اختيار غير صحيح");`	يُطبع تنبيه للمستخدم بأن ما كتبه ليس من الاختيارات المتاحة، ثم break الأخيرة تُنهي الـ switch.

التمارين العملية

تمرين 1: زوجي أم فردي

سهل

اقرأ عددًا واطبع هل هو زوجي أم فردي.

الحل المقترح:

```C#
Console.Write("أدخل عددًا: ");
int n = int.Parse(Console.ReadLine());
if (n % 2 == 0)
    Console.WriteLine("زوجي");
else
    Console.WriteLine("فردي");
```

ماذا يفعل هذا الكود؟ يطلب البرنامج عددًا صحيحًا ثم يطبع «زوجي» إذا كان يقبل القسمة على 2، أو «فردي» إذا لم يكن كذلك.

`Console.Write("أدخل عددًا: ");`	نطبع طلبًا للمستخدم ليكتب عددًا.
`int n = int.Parse(Console.ReadLine());`	نقرأ ما كتبه كنص بـ ReadLine، ثم نحوّله إلى عدد صحيح بـ int.Parse ونحفظه في n.
`if (n % 2 == 0)`	n % 2 يحسب باقي قسمة العدد على 2، والعلامة == تعني «هل يساوي؟» للمقارنة، وهي تختلف عن = التي تضع قيمة في متغير. فإذا كان الباقي صفرًا فالعدد زوجي.
`Console.WriteLine("زوجي");`	يُطبع «زوجي» فقط عندما يكون الشرط صحيحًا.

`else`	else تعني «وإلا»: تُنفذ عندما يكون شرط if خاطئًا، أي عندما يكون الباقي 1 (أو -1 للأعداد السالبة).
`Console.WriteLine("فردي");`	يُطبع «فردي» في هذه الحالة.

تمرين 2: تقدير الطالب

متوسط

اقرأ درجة من 100. إن كانت أقل من 0 أو أكبر من 100 اطبع «درجة غير صالحة»، وإلا اطبع التقدير: ممتاز (90+)، جيد جدًا (80+)، جيد (70+)، مقبول (60+)، راسب.

الحل المقترح:

C#

```
Console.Write("الدرجة: ");
int s = int.Parse(Console.ReadLine());

if (s < 0 || s > 100) Console.WriteLine("درجة غير صالحة");
else if (s ⩾ 90) Console.WriteLine("ممتاز");
else if (s ⩾ 80) Console.WriteLine("جيد جدًا");
else if (s ⩾ 70) Console.WriteLine("جيد");
else if (s ⩾ 60) Console.WriteLine("مقبول");
else Console.WriteLine("راسب");
```

ماذا يفعل هذا الكود؟ يطلب البرنامج درجة من 0 إلى 100 ويطبع تقديرها (ممتاز، جيد جدًا، جيد، مقبول، راسب)، ويطبع «درجة غير صالحة» إذا كانت الدرجة خارج هذا المدى.

`Console.Write("الدرجة: ");`	نطبع طلبًا للمستخدم ليكتب الدرجة.
`int s = int.Parse(Console.ReadLine());`	نقرأ النص الذي كتبه ونحوّله إلى عدد صحيح بـ int.Parse، ونحفظه في المتغير s.
`if (s < 0 || s > 100) Console.WriteLine("درجة غير صالحة");`	العلامة || تعني «أو»: يكون الشرط صحيحًا إذا تحقق أحد الجزأين على الأقل، أي الدرجة أقل من 0 أو أكبر من 100. نفحص هذا أولًا لنرفض الدرجات غير المنطقية، ويمكن كتابة التعليمة في نفس سطر if.
`else if (s ⩾ 90) Console.WriteLine("ممتاز");`	نصل إلى هنا فقط إذا كانت الدرجة صالحة. إذا كانت 90 أو أكثر يُطبع «ممتاز».
`else if (s ⩾ 80) Console.WriteLine("جيد جدًا");`	إذا فشلت الشروط السابقة وكانت الدرجة 80 أو أكثر (أي 80 إلى 89) يُطبع «جيد جدًا».
`else if (s ⩾ 70) Console.WriteLine("جيد");`	الدرجات من 70 إلى 79 تعطي «جيد». ترتيب الشروط من الأعلى إلى الأدنى مهم لأن أول شرط صحيح هو الذي يُنفذ.

`else if (s ⩾ 60) Console.WriteLine("مقبول");`	الدرجات من 60 إلى 69 تعطي «مقبول».
`else Console.WriteLine("راسب");`	else الأخيرة تلتقط كل ما تبقى، أي الدرجات من 0 إلى 59، فيُطبع «راسب».

تمرين 3: تحدٍّ: آلة حاسبة

تحدٍّ

اقرأ عددين وعملية (+ أو - أو * أو /) واستخدم switch لطباعة الناتج. لا تسمح بالقسمة على صفر.

الحل المقترح:

C#

```
Console.Write("العدد الأول: ");
double x = double.Parse(Console.ReadLine());
Console.Write("العملية (+ - * /): ");
string op = Console.ReadLine();
Console.Write("العدد الثاني: ");
double y = double.Parse(Console.ReadLine());

switch (op)
{
    case "+": Console.WriteLine(x + y); break;
    case "-": Console.WriteLine(x - y); break;
    case "*": Console.WriteLine(x * y); break;
    case "/":
        if (y == 0) Console.WriteLine("لا يمكن القسمة على صفر");
        else Console.WriteLine(x / y);
        break;
    default: Console.WriteLine("عملية غير معروفة"); break;
}
```

ماذا يفعل هذا الكود؟ آلة حاسبة بسيطة: يطلب البرنامج عددين وعملية حسابية (+ أو - أو * أو /) ثم يطبع النتيجة، ويرفض القسمة على صفر أو أي عملية غير معروفة.

`Console.Write("العدد الأول: ");`	نطبع طلبًا للمستخدم ليكتب العدد الأول.
`double x = double.Parse(Console.ReadLine());`	نقرأ النص ونحوّله إلى عدد عشري بـ double.Parse حتى نقبل أعدادًا بكسور، ونحفظه في x.
`Console.Write("العملية (+ - * /): ");`	نطلب من المستخدم أن يكتب رمز العملية التي يريدها.

`string op = Console.ReadLine();`	نحفظ رمز العملية كنص في المتغير op، ولا نحوّله لأنه رمز وليس عددًا.
`Console.Write("العدد الثاني: ");`	نطلب العدد الثاني.
`double y = double.Parse(Console.ReadLine());`	نقرأ العدد الثاني ونحوّله إلى double ونحفظه في y.
`switch (op)`	switch يقارن قيمة op بكل حالة case، وينفذ الحالة التي تطابق رمز العملية.
`{`	الأقواس المعقوفة { } تجمع التعليمات في كتلة واحدة، وهنا تحدد بداية ونهاية جسم الـ switch.
`case "+": Console.WriteLine(x + y); break;`	إذا كان op هو "+" نطبع ناتج الجمع ثم break للخروج من الـ switch. يمكن كتابة case وتعليماتها و break في سطر واحد.
`case "-": Console.WriteLine(x - y); break;`	إذا كان op هو "-" نطبع ناتج الطرح ثم نخرج.
`case "*": Console.WriteLine(x * y); break;`	إذا كان op هو "*" نطبع ناتج الضرب ثم نخرج.
`case "/":`	حالة القسمة تحتاج عدة أسطر لأننا نريد فحص شيء قبل القسمة.
`if (y == 0) Console.WriteLine("لا يمكن القسمة على صفر");`	نستخدم == للمقارنة: إذا كان العدد الثاني صفرًا نطبع رسالة تحذير. مع النوع double لا يتوقف البرنامج عند القسمة على صفر بل يعطي قيمة غريبة مثل ∞ (Infinity)، لذلك نمنعها بأنفسنا.
`else Console.WriteLine(x / y);`	إذا لم يكن y صفرًا نقسم ونطبع النتيجة بأمان.
`break;`	break تُنهي حالة القسمة وتخرج من الـ switch، وهي ضرورية في C# في نهاية كل case.
`default: Console.WriteLine("عملية غير معروفة"); break;`	default تُنفذ إذا كتب المستخدم رمزًا لا يطابق أي حالة، مثل % أو حرف، فنخبره أن العملية غير معروفة.

اختبر نفسك (3 أسئلة)

1. ما ناتج 10 % 3 ؟

3 | 1 | 0

2. لكي يتحقق الشرط يجب أن يكون الشرطان صحيحين، نستخدم:

|| && !

3. ما الكلمة التي تنهي كل حالة case؟

stop end break

الإجابات: 1) 1 · 2) && · 3) break

الواجب: راجع دروس الأسبوع الأول، واختر فكرة مشروعك النهائي من قائمة المشروع.

C# اليوم 6 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

الحلقات التكرارية والمصفوفات

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يكرر الأوامر باستخدام for وwhile.
- يميز متى يستخدم for ومتى يستخدم while.
- يخزن مجموعة قيم في مصفوفة ويصل إليها بالفهرس.
- يمر على المصفوفة بـ foreach ويحسب المجموع والمتوسط والأكبر.

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. حلقة for

نستخدمها عندما نعرف عدد مرات التكرار مسبقًا. تتكون من: **البداية**؛ **شرط الاستمرار**؛ **الزيادة**.

C#

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"المحاولة رقم {i}");
}
```

Output ▸

```
المحاولة رقم 1
المحاولة رقم 2
المحاولة رقم 3
المحاولة رقم 4
المحاولة رقم 5
```

ماذا يفعل هذا الكود؟ برنامج يكرر طباعة سطر واحد خمس مرات باستخدام حلقة for، فيرى الطالب: المحاولة رقم 1 حتى المحاولة رقم 5.

`for (int i = 1; i ⩽ 5; i++)`	الكلمة for تبدأ حلقة تكرار لها ثلاثة أجزاء مفصولة بفاصلة منقوطة: البداية int i = 1 تنشئ عدّادًا قيمته 1، والشرط i <= 5 يعني استمر ما دام العداد أصغر من أو يساوي 5، والخطوة i++ تزيد العداد بمقدار 1 بعد كل دورة.
`{`	الأقواس المعقوصة { و } تجمع مجموعة أسطر في كتلة واحدة، وهنا تحدد الأسطر التي ستتكرر في كل دورة من الحلقة.
`Console.WriteLine($"المحاولة رقم {i}");`	يطبع هذا السطر نصًا ثم ينتقل لسطر جديد. العلامة $ قبل النص تسمح بوضع متغير داخل أقواس { } فتظهر قيمته، لذلك يُطبع رقم الدورة الحالية: 1 ثم 2 ثم ... حتى 5.

2. حلقة while

تستمر ما دام الشرط صحيحًا، وتناسب الحالات التي لا نعرف فيها عدد التكرارات، مثل قوائم البرامج.

C#

```
string password = "";
while (password ≠ "1234")
{
    Console.Write("أدخل كلمة المرور: ");
    password = Console.ReadLine();
}
Console.WriteLine("تم الدخول بنجاح");
```

ماذا يفعل هذا الكود؟ برنامج يطلب كلمة المرور مرارًا حتى يكتب المستخدم 1234، ثم يطبع رسالة نجاح الدخول.

`string password = "";`	ننشئ متغيرًا نصيًا من نوع string اسمه password ونعطيه قيمة فارغة "" في البداية، حتى يكون له قيمة يمكن فحصها في شرط الحلقة.
`while (password ≠ "1234")`	الحلقة while تكرر الكتلة التي بعدها ما دام الشرط صحيحًا. المعامل != يعني «لا يساوي»، أي استمر في التكرار ما دامت كلمة المرور ليست "1234". نستخدم while عندما لا نعرف مسبقًا عدد مرات التكرار.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر داخل الحلقة في كتلة واحدة.
`Console.Write("أدخل كلمة المرور: ");`	الأمر Console.Write يطبع النص دون أن ينتقل إلى سطر جديد، فيكتب المستخدم إجابته بجانب السؤال مباشرة.

`password = Console.ReadLine();`	الأمر Console.ReadLine ينتظر المستخدم حتى يكتب نصًا ويضغط Enter، ثم نخزّن ما كتبه في المتغير password لتُفحص قيمته الجديدة في شرط الحلقة مرة أخرى.
`Console.WriteLine("تم الدخول بنجاح");`	هذا السطر خارج الحلقة، فلا يُنفَّذ إلا بعد أن يصبح الشرط خاطئًا، أي بعد أن يكتب المستخدم 1234 فتنتهي الحلقة.

احذر الحلقة اللانهائية: تأكد أن شيئًا داخل الحلقة يغير الشرط في النهاية.

3. المصفوفات

المصفوفة تخزن عدة قيم من نفس النوع. الفهرس يبدأ من **0**، وعدد العناصر `Length` .

C#

```
int[] grades = { 85, 92, 78, 95, 60, 88 };

Console.WriteLine(grades[0]);       // 85
Console.WriteLine(grades.Length);   // 6

int sum = 0;
int max = grades[0];
foreach (int g in grades)
{
    sum += g;
    if (g > max) max = g;
}
double avg = (double)sum / grades.Length;
Console.WriteLine($"المتوسط: {avg:F1} - الأعلى: {max}");
```

```
Output ▸ 85
6
95 :الأعلى - 83.0 :المتوسط
```

ماذا يفعل هذا الكود؟ برنامج يخزّن ست درجات في مصفوفة، ثم يطبع أول درجة وعدد الدرجات، ويحسب المتوسط وأعلى درجة ويطبعهما.

`int[] grades = { 85, 92, 78, 95, 60, 88 };`	int[] تعني مصفوفة (array) من الأعداد الصحيحة، والمصفوفة متغير واحد يحمل عدة قيم مرتبة. نضع القيم الست بين { } مفصولة بفواصل لتكون القيم الأولى للمصفوفة grades.
`Console.WriteLine(grades[0]); // 85`	نصل إلى عنصر في المصفوفة برقم موقعه بين أقواس مربعة [ ]، ويسمى الفهرس (index). الترقيم يبدأ من 0 لا من 1، لذلك grades[0] هو أول عنصر وقيمته 85. والنص بعد // تعليق يتجاهله الحاسوب.

`Console.WriteLine(grades.Length); // 6`	الخاصية .Length تعطي عدد عناصر المصفوفة، وهنا 6. لاحظ أن آخر فهرس يكون دائمًا Length - 1 أي 5.
`int sum = 0;`	ننشئ متغيرًا sum يبدأ بصفر لنجمع فيه الدرجات واحدة تلو الأخرى.
`int max = grades[0];`	نفترض في البداية أن أعلى درجة هي أول عنصر، ثم سنقارن بقية الدرجات بها ونحدّثها إذا وجدنا درجة أكبر.
`foreach (int g in grades)`	الحلقة foreach تمر على كل عنصر في المصفوفة بالترتيب دون الحاجة إلى عدّاد أو فهرس، وفي كل دورة يحمل المتغير g قيمة العنصر الحالي.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تُنفَّذ في كل دورة من الحلقة في كتلة واحدة.
`sum += g;`	المعامل += يعني «أضف إلى القيمة الحالية»، فهذا السطر يساوي sum = sum + g، أي نضيف الدرجة الحالية إلى المجموع.
`if (g > max) max = g;`	إذا كانت الدرجة الحالية g أكبر من أعلى درجة وجدناها حتى الآن، نجعلها هي الأعلى. وعندما يكون بعد if أمر واحد فقط يمكن كتابته دون أقواس { }.
`double avg = (double)sum / grades.Length;`	نحسب المتوسط بقسمة المجموع على عدد الدرجات. الكتابة (double) قبل sum تحوّله إلى عدد عشري (تحويل النوع أو cast)، لأن قسمة عددين صحيحين في C# تحذف الكسور، فنحتاج العشري لنحصل على نتيجة دقيقة.
`Console.WriteLine($"المتوسط: {avg:F1} - الأعلى: {max}");`	نطبع المتوسط وأعلى درجة داخل نص واحد. الكتابة :F1 بعد avg تعني «اعرض الرقم برقم عشري واحد بعد الفاصلة»، فيظهر المتوسط 83.0 بدل رقم طويل.

التمارين العملية

تمرين 1: جدول الضرب

سهل

اقرأ رقمًا واطبع جدول ضربه من 1 إلى 10.

الحل المقترح:

C#

```
Console.Write("الرقم: ");
int n = int.Parse(Console.ReadLine());
for (int i = 1; i ⩽ 10; i++)
    Console.WriteLine($"{n} × {i} = {n * i}");
```

ماذا يفعل هذا الكود؟ برنامج يطلب رقمًا من المستخدم ثم يطبع جدول ضربه من 1 إلى 10، كل عملية في سطر.

`Console.Write("الرقم: ");`	نطبع سؤالًا للمستخدم دون الانتقال إلى سطر جديد، ليكتب الرقم بجانبه.
`int n = int.Parse(Console.ReadLine());`	Console.ReadLine يقرأ ما يكتبه المستخدم كنص، ثم int.Parse يحوّل هذا النص إلى عدد صحيح لنتمكن من الحساب به، ونخزّنه في n. إذا كتب المستخدم شيئًا ليس رقمًا يتوقف البرنامج بخطأ.
`for (int i = 1; i ⩽ 10; i++)`	حلقة for تبدأ العداد i من 1، وتستمر ما دام i <= 10، وتزيده بواحد في كل دورة عبر i++، فتتكرر 10 مرات. لا توجد أقواس { } لأن جسم الحلقة أمر واحد فقط هو السطر التالي.
`Console.WriteLine($"{n} × {i} = {n * i}");`	يطبع سطرًا من جدول الضرب مثل 7 × 3 = 21. داخل { } يمكن وضع عملية حسابية أيضًا، فالتعبير {n * i} يُحسب أولًا ثم تُعرض نتيجته؛ والرمز * هو الضرب في C#، أما × فهو مجرد نص للعرض.

تمرين 2: درجات الطلاب الستة

متوسط

اقرأ درجات 6 طلاب في مصفوفة، ثم اطبع المتوسط وأعلى درجة وأقل درجة وعدد الناجحين (60 فأكثر).

الحل المقترح:

C#

```
int[] g = new int[6];
for (int i = 0; i < g.Length; i++)
{
    Console.Write($"درجة الطالب {i + 1}: ");
    g[i] = int.Parse(Console.ReadLine());
}

int sum = 0, max = g[0], min = g[0], passed = 0;
foreach (int x in g)
{
    sum += x;
    if (x > max) max = x;
    if (x < min) min = x;
    if (x ⩾ 60) passed++;
}
Console.WriteLine($"المتوسط: {(double)sum / g.Length:F1}");
Console.WriteLine($"الأعلى: {max} - الأقل: {min} - الناجحون: {passed}");
```

ماذا يفعل هذا الكود؟ برنامج يطلب درجات ستة طلاب ويخزّنها في مصفوفة، ثم يطبع المتوسط وأعلى درجة وأقل درجة وعدد الناجحين.

`int[] g = new int[6];`	ننشئ مصفوفة أعداد صحيحة اسمها g تتسع لـ6 عناصر. الكلمة new تحجز مكانًا جديدًا في الذاكرة، وتبدأ كل العناصر بقيمة 0 حتى نملأها.

`for (int i = 0; i < g.Length; i++)`	حلقة for يبدأ عدادها من 0 لأن فهارس المصفوفة تبدأ من 0، وتستمر ما دام i < g.Length أي حتى 5، فتمر على المواقع الستة كلها.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر في كل دورة من الحلقة في كتلة واحدة.
`Console.Write($"درجة الطالب {i + 1}: ");`	نطبع سؤالًا يحمل رقم الطالب. نكتب i + 1 لأن العداد يبدأ من 0، والناس يعدّون من 1، فيظهر «درجة الطالب 1» بدل 0.
`g[i] = int.Parse(Console.ReadLine());`	نقرأ ما يكتبه المستخدم ونحوّله إلى عدد صحيح بـ int.Parse، ثم نضعه في الموقع رقم i من المصفوفة.
`int sum = 0, max = g[0], min = g[0], passed = 0;`	نعرّف أربعة متغيرات من النوع int في سطر واحد مفصولة بفواصل: sum للمجموع، وmax وmin يبدآن بأول درجة لنقارن بها، وpassed لعدّ الناجحين.
`foreach (int x in g)`	الحلقة foreach تمر على كل درجة في المصفوفة g بالترتيب، ويحمل المتغير x الدرجة الحالية في كل دورة.
`sum += x;`	المعامل += يضيف x إلى قيمة sum الحالية، أي sum = sum + x.
`if (x > max) max = x;`	إذا كانت الدرجة الحالية أكبر من أعلى درجة حتى الآن، تصبح هي الأعلى الجديدة.
`if (x < min) min = x;`	وبالمثل، إذا كانت الدرجة الحالية أصغر من أقل درجة حتى الآن، تصبح هي الأقل الجديدة.
`if (x ⩾ 60) passed++;`	المعامل >= يعني «أكبر من أو يساوي»؛ فإذا كانت الدرجة 60 أو أكثر نزيد عداد الناجحين بواحد باستخدام ++.
`Console.WriteLine($"المتوسط: {(double)sum / g.Length:F1}");`	نحسب المتوسط مباشرة داخل النص: (double) تحوّل المجموع إلى عدد عشري حتى لا تُحذف الكسور عند القسمة، و:F1 تعرض النتيجة برقم عشري واحد بعد الفاصلة.
`Console.WriteLine($"الأعلى: {max} - الأقل: {min} - الناجحون: {passed}");`	نطبع أعلى درجة وأقل درجة وعدد الناجحين في سطر واحد، كل قيمة داخل { } تُستبدل بقيمة متغيرها.

تمرين 3: تحدٍّ: لعبة تخمين الرقم

تحدٍّ

يختار البرنامج رقمًا عشوائيًا من 1 إلى 100، ويخمّن المستخدم حتى يصيب. بعد كل محاولة اطبع «أكبر» أو «أصغر»، وفي النهاية اطبع عدد المحاولات.

الحل المقترح:

C#

```
int secret = Random.Shared.Next(1, 101);
int guess = 0, tries = 0;

while (guess ≠ secret)
{
    Console.Write("خمّن: ");
    guess = int.Parse(Console.ReadLine());
    tries++;
    if (guess < secret) Console.WriteLine("أكبر ⬆");
    else if (guess > secret) Console.WriteLine("أصغر ⬇");
}
Console.WriteLine($"أحسنت! عرفته في {tries} محاولات");
```

ماذا يفعل هذا الكود؟ لعبة تخمين: يختار البرنامج رقمًا سريًا عشوائيًا من 1 إلى 100، والمستخدم يخمّن ويحصل على تلميح «أكبر» أو «أصغر» حتى يصيب، ثم يرى عدد محاولاته.

`int secret = Random.Shared.Next(1, 101);`	Random.Shared مولّد أرقام عشوائية جاهز في .NET، والدالة Next(1, 101) تعطي عددًا صحيحًا عشوائيًا من 1 حتى 100؛ لأن الحد الثاني 101 غير داخل في النطاق. نخزّن الرقم في secret.
`int guess = 0, tries = 0;`	نعرّف متغيرين: guess لتخمين المستخدم ويبدأ بـ0 (رقم لا يمكن أن يكون هو السر)، وtries لعدّ المحاولات ويبدأ بـ0.
`while (guess ≠ secret)`	الحلقة while تستمر ما دام التخمين لا يساوي (!=) الرقم السري، لأننا لا نعرف مسبقًا كم محاولة سيحتاج المستخدم.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر في كل محاولة في كتلة واحدة.
`Console.Write("خمّن: ");`	نطلب من المستخدم أن يكتب تخمينه بجانب الكلمة دون سطر جديد.
`guess = int.Parse(Console.ReadLine());`	نقرأ ما كتبه المستخدم ونحوّله من نص إلى عدد صحيح بـ int.Parse، ونخزّنه في guess.
`tries++;`	المعامل ++ يزيد قيمة tries بمقدار 1، فنحسب هذه المحاولة.
`if (guess < secret) Console.WriteLine("أكبر ⬆");`	إذا كان التخمين أصغر من الرقم السري، نخبر المستخدم أن الرقم المطلوب أكبر.
`else if (guess > secret) Console.WriteLine("أصغر ⬇");`	else if تعني «وإلا، إذا...»: تُفحص فقط إذا لم يتحقق الشرط السابق؛ فإذا كان التخمين أكبر من السر نخبره أن الرقم أصغر. وإذا كان مساويًا لا يُطبع شيء وتنتهي الحلقة.

```
Console.WriteLine($"أحسنت! عرفته في {tries} محاولات");
```

بعد الخروج من الحلقة، أي عند التخمين الصحيح، نطبع رسالة تهنئة مع عدد المحاولات المخزّن في tries.

اختبر نفسك (3 أسئلة)

1. كم مرة تُنفذ: for (int i = 0; i < 3; i++)؟

2 | 3 | 4

2. ما فهرس أول عنصر في المصفوفة؟

0 | 1 | 1-

3. أي حلقة تناسب «كرر حتى يكتب المستخدم 0»؟

for | while | لا شيء

الإجابات: 1) 3 · 2) 0 · 3) while

الواجب: إعلان المشروع النهائي: اقرأ متطلبات المشروع وابدأ بصفحة HTML الخاصة بفكرتك.

C# اليوم 7 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

الدوال والقوائم List ومدخل للكائنات

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يقسّم البرنامج إلى دوال (Methods) تستقبل مدخلات وتُرجع نتيجة.
- يستخدم List لإضافة العناصر وحذفها بمرونة.
- يعرّف كلاسًا بسيطًا Student بخصائص وينشئ منه كائنات.
- يتعامل مع الإدخال الخاطئ باستخدام int.TryParse.

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. الدوال (Methods)

الدالة مجموعة أوامر لها اسم، نكتبها مرة ونستدعيها كثيرًا. `void` تعني أنها لا تُرجع قيمة.

```C#
static void Greet(string name)
{
    Console.WriteLine($"يا أهلًا {name}");
}

static bool IsPassed(int score)
{
    return score ⩾ 60;
}

Greet("مريم");
Console.WriteLine(IsPassed(75));   // True
Console.WriteLine(IsPassed(40));   // False
```

ماذا يفعل هذا الكود؟ برنامج يعرّف دالتين: الأولى تطبع ترحيبًا بالاسم، والثانية تفحص هل الدرجة ناجحة. يرى الطالب «أهلًا يا مريم» ثم True ثم False.

`static void Greet(string name)`	هنا نعرّف دالة (method) اسمها Greet، والدالة مجموعة أوامر لها اسم نستدعيها متى شئنا. void تعني أنها لا تُرجع أي نتيجة، وما بين القوسين (string name) هو مُدخَل (parameter) نصي تستقبله. وstatic تعني أنها دالة مستقلة لا تستخدم متغيرات من خارجها.
`{`	الأقواس المعقوصة { و } تجمع أوامر الدالة في كتلة واحدة، فكل ما بينها هو جسم الدالة.
`Console.WriteLine($"أهلًا يا {name}");`	هذا هو عمل الدالة: تطبع ترحيبًا يحتوي على الاسم الذي أُرسل إليها في name.
`static bool IsPassed(int score)`	دالة ثانية اسمها IsPassed تستقبل عددًا صحيحًا score. النوع bool قبل اسمها يعني أنها تُرجع نتيجة منطقية: true (صحيح) أو false (خطأ).
`return score ⩾ 60;`	الكلمة return تُنهي الدالة وتُعيد النتيجة لمن استدعاها. هنا نُرجع ناتج المقارنة score >= 60 مباشرة، فيكون true إذا كانت الدرجة 60 أو أكثر وإلا false.
`Greet("مريم");`	نستدعي (نشغّل) الدالة Greet ونرسل لها النص "مريم"، فيصبح name داخلها يساوي مريم ويُطبع «أهلًا يا مريم».
`Console.WriteLine(IsPassed(75));` `// True`	نستدعي IsPassed بالدرجة 75، فتُرجع true لأن 75 >= 60، ثم نطبع النتيجة فتظهر True.
`Console.WriteLine(IsPassed(40));` `// False`	هذه المرة الدرجة 40 أقل من 60، فتُرجع الدالة false ويظهر False. هكذا نعيد استخدام الدالة نفسها بقيم مختلفة.

2. القوائم List

المصفوفة حجمها ثابت، أما `List` فتكبر وتصغر. أهم العمليات: `Add` و`Remove` و`Count` .

C#

```
List<string> names = new List<string>();
names.Add("سارة");
names.Add("خالد");
names.Add("مريم");
names.Remove("خالد");

Console.WriteLine($"عدد الطلاب: {names.Count}");
foreach (string n in names)
    Console.WriteLine("- " + n);
```

```
Output ▸ 2 :عدد الطلاب
- سارة
- مريم
```

ماذا يفعل هذا الكود؟ برنامج ينشئ قائمة أسماء، يضيف إليها ثلاثة أسماء ثم يحذف واحدًا، ويطبع عدد الطلاب (2) والأسماء المتبقية.

`List<string> names = new List<string>();`	List<string> قائمة من النصوص، وتختلف عن المصفوفة بأن حجمها يكبر ويصغر عند الإضافة والحذف. ما بين < > هو نوع العناصر، والكلمة new تنشئ قائمة جديدة فارغة.
`names.Add("سارة");`	الدالة Add تضيف عنصرًا إلى آخر القائمة؛ وهذا السطر والسطران التاليان يضيفون سارة ثم خالد ثم مريم بالترتيب.
`names.Remove("خالد");`	الدالة Remove تبحث عن العنصر "خالد" وتحذفه من القائمة، فتتقدم العناصر التي بعده مكانه.
`Console.WriteLine($"عدد الطلاب: {names.Count}");`	الخاصية Count تعطي عدد العناصر الموجودة الآن في القائمة، وهي 2 بعد الحذف. (في المصفوفة نستخدم Length، وفي List نستخدم Count).
`foreach (string n in names)`	foreach تمر على كل اسم في القائمة بالترتيب، ويحمل المتغير n الاسم الحالي. جسم الحلقة هو السطر التالي فقط لذلك لا نحتاج { }.
`Console.WriteLine("- " + n);`	المعامل + مع النصوص يلصقها معًا، فيُطبع كل اسم مسبوقًا بشرطة مثل «- سارة».

3. الكلاس والكائن

الكلاس **قالب** يصف شيئًا (مثل الطالب)، والكائن **نسخة** فعلية منه. هذا هو أساس البرمجة الكائنية، وسنستخدمه لتمثيل صفوف قاعدة البيانات.

C#

```
var list = new List<Student>
{
    new Student { Name = "سارة", Grade = 95 },
    new Student { Name = "خالد", Grade = 55 }
};

foreach (var s in list)
    Console.WriteLine($"{s.Name}: {s.Grade} - {(s.Grade ⩾ 60 ? "راسب" : "ناجح")}");

class Student
{
    public string Name { get; set; }
    public int Grade { get; set; }
```

```
}
```

ماذا يفعل هذا الكود؟ برنامج يعرّف صنفًا Student له اسم ودرجة، وينشئ قائمة بطالبين، ثم يطبع لكل طالب اسمه ودرجته وهل هو ناجح أم راسب.

`var list = new List<Student>`	ننشئ قائمة عناصرها من النوع Student (صنف سنعرّفه في آخر الملف). الكلمة var تجعل #C يستنتج نوع المتغير تلقائيًا من الجهة اليمنى، فلا نحتاج كتابة List<Student> مرتين.
`{`	الأقواس المعقوصة هنا تجمع العناصر الأولى التي نضعها في القائمة لحظة إنشائها، مفصولة بفواصل.
`new Student { Name = "سارة", Grade = 95 },`	new Student ينشئ كائنًا (object) جديدًا من الصنف Student، أي طالبًا فعليًا، وما بين { } يعطي خصائصه قيمها الأولى: الاسم سارة والدرجة 95.
`new Student { Name = "خالد", Grade = 55 }`	ننشئ طالبًا ثانيًا اسمه خالد ودرجته 55. لا توجد فاصلة بعده لأنه آخر عنصر في القائمة.
`foreach (var s in list)`	نمر على كل طالب في القائمة، ويحمل المتغير s الطالب الحالي؛ وvar هنا تعني أن نوعه Student.
`Console.WriteLine($"{s.Name}: {s.Grade} - {(s.Grade >= 60 ? "ناجح" : "راسب")}");`	النقطة في s.Name تعني «خاصية Name الخاصة بهذا الطالب». أما الشرط المختصر ? : فيعمل هكذا: إذا كان s.Grade >= 60 صحيحًا يعطي "ناجح" وإلا يعطي "راسب"، ونضعه بين أقواس ( ) داخل { } لأنه يحتوي على النقطتين :.
`class Student`	الكلمة class تعرّف صنفًا، وهو قالب أو مخطط يصف شكل نوع جديد من البيانات؛ هنا كل طالب له اسم ودرجة. في #C يجب أن تأتي تعريفات الأصناف بعد أوامر البرنامج الرئيسية في الملف.
`public string Name { get; set; }`	هذه خاصية (property) اسمها Name من النوع string. الكلمة public تعني أنه يمكن الوصول إليها من خارج الصنف، و{ get; set; } تعني أنه يمكن قراءة قيمتها (get) وتغييرها (set).
`public int Grade { get; set; }`	خاصية ثانية اسمها Grade من النوع int لتخزين درجة الطالب، ويمكن قراءتها وتغييرها أيضًا.

في مشاريع top-level statements يكون تعريف الكلاس في نهاية الملف بعد الأوامر.

4. حماية البرنامج من الإدخال الخاطئ

إذا كتب المستخدم حروفًا بدل رقم، فإن `int.Parse` يوقف البرنامج بخطأ. `int.TryParse` يحاول التحويل ويُرجع true أو false.

```
C#
```

```
Console.Write("العمر: ");
if (int.TryParse(Console.ReadLine(), out int age))
    Console.WriteLine($"عمرك {age}");
else
    Console.WriteLine("الرجاء إدخال رقم صحيح");
```

ماذا يفعل هذا الكود؟ برنامج يطلب العمر ويتأكد أن المدخل رقم صحيح بأمان: إن كان رقمًا يطبع «عمرك ...»، وإلا يطبع رسالة تطلب إدخال رقم صحيح بدل أن يتوقف بخطأ.

`Console.Write("العمر: ");`	نطبع سؤال العمر دون الانتقال إلى سطر جديد.
`if (int.TryParse(Console.ReadLine(), out int age))`	int.TryParse «تحاول» تحويل النص إلى عدد صحيح: تُرجع true إذا نجحت وfalse إذا فشلت، دون أن يتوقف البرنامج كما يحدث مع int.Parse. والكلمة out int age تعني أن الدالة ستضع الرقم الناتج في متغير جديد اسمه age ننشئه في المكان نفسه.
`Console.WriteLine($"عمرك {age}");`	هذا السطر يُنفَّذ فقط إذا نجح التحويل، فنطبع العمر المخزّن في age.
`else`	else تعني «وإلا»: الأمر الذي بعدها يُنفَّذ عندما يكون شرط if خاطئًا، أي عندما يكتب المستخدم شيئًا ليس رقمًا صحيحًا.
`Console.WriteLine("الرجاء إدخال رقم صحيح");`	نطبع رسالة لطيفة تخبر المستخدم بالخطأ بدلًا من أن ينهار البرنامج.

التمارين العملية

تمرين 1: دالة المساحة

سهل

اكتب دالة CircleArea تستقبل نصف القطر وتُرجع مساحة الدائرة (Math.PI × r × r)، واستدعها لثلاث قيم.

الحل المقترح:

C#

```
static double CircleArea(double r)
{
    return Math.PI * r * r;
}

Console.WriteLine(CircleArea(1).ToString("F2"));
Console.WriteLine(CircleArea(2.5).ToString("F2"));
Console.WriteLine(CircleArea(10).ToString("F2"));
```

ماذا يفعل هذا الكود؟ برنامج يعرّف دالة تحسب مساحة الدائرة من نصف قطرها، ثم يطبع مساحة ثلاث دوائر مقرّبة إلى

رقمين عشريين: 3.14 و19.63 و314.16.

`static double CircleArea(double r)`	نعرّف دالة اسمها CircleArea تستقبل نصف القطر r كعدد عشري (double)، وتُرجع نتيجة من النوع double أيضًا. الكلمة static تعني أنها دالة مستقلة لا تعتمد على متغيرات خارجها.
`{`	الأقواس المعقوصة { و } تجمع أوامر الدالة في كتلة واحدة هي جسم الدالة.
`return Math.PI * r * r;`	Math.PI ثابت جاهز في C# قيمته π تقريبًا 3.14159، ومساحة الدائرة = π × نصف القطر × نصف القطر. الكلمة return تُرجع هذه النتيجة لمن استدعى الدالة.
`Console.WriteLine(CircleArea(1).ToString("F2"));`	نستدعي الدالة بنصف قطر 1، ثم الدالة ToString("F2") تحوّل الرقم الناتج إلى نص برقمين بعد الفاصلة العشرية، فيظهر 3.14.
`Console.WriteLine(CircleArea(2.5).ToString("F2"));`	نعيد استخدام الدالة نفسها بنصف قطر 2.5، فيظهر 19.63. هذه فائدة الدوال: نكتب الحساب مرة ونستخدمه مرات.
`Console.WriteLine(CircleArea(10).ToString("F2"));`	ومرة ثالثة بنصف قطر 10، فيظهر 314.16.

تمرين 2: قائمة طلاب تفاعلية

متوسط

برنامج بقائمة: 1 إضافة اسم، 2 عرض الأسماء، 3 حذف اسم، 0 خروج. استخدم while وswitch وList.

الحل المقترح:

C#

```
var names = new List<string>();
string choice = "";

while (choice ≠ "0")
{
    Console.WriteLine("\n1 خروج 0 | حذف 3 | عرض 2 | إضافة");
    choice = Console.ReadLine();
    switch (choice)
    {
        case "1":
            Console.Write("الاسم: ");
            names.Add(Console.ReadLine());
            break;
        case "2":
            for (int i = 0; i < names.Count; i++)
                Console.WriteLine($"{i + 1}. {names[i]}");
            break;
        case "3":
```

```
            Console.Write("الاسم المراد حذفه: ");
            if (names.Remove(Console.ReadLine())) Console.WriteLine("تم الحذف");
            else Console.WriteLine("غير موجود");
            break;
    }
}
```

ماذا يفعل هذا الكود؟ برنامج قائمة أسماء تفاعلي يعرض قائمة خيارات متكررة: إضافة اسم، عرض الأسماء مرقّمة، حذف اسم، أو الخروج بكتابة 0.

`var names = new List<string>();`	ننشئ قائمة نصوص فارغة باسم names لتخزين الأسماء، وvar تجعل C# يستنتج أن نوعها List<string>.
`string choice = "";`	متغير نصي يحفظ اختيار المستخدم من القائمة، ونبدؤه بنص فارغ حتى تدخل الحلقة أول مرة.
`while (choice ≠ "0")`	الحلقة while تستمر في عرض القائمة ما دام المستخدم لم يختر "0" (الخروج).
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر داخل الحلقة في كتلة واحدة.
`Console.WriteLine("\n1 2 | إضافة 3 | حذف 0 | خروج");`	نطبع قائمة الخيارات. الرمز \n داخل النص يعني «سطر جديد»، فيترك سطرًا فارغًا قبل القائمة لتكون أوضح.
`choice = Console.ReadLine();`	نقرأ اختيار المستخدم كنص ونخزّنه في choice.
`switch (choice)`	switch يقارن قيمة choice بعدة احتمالات، وكل احتمال يبدأ بالكلمة case؛ وهو بديل أوضح من كتابة if و else if كثيرة.
`case "1":`	هذا الجزء يُنفَّذ إذا كانت قيمة choice تساوي "1" أي اختيار الإضافة.
`Console.Write("الاسم: ");`	نطلب من المستخدم كتابة الاسم المراد إضافته.
`names.Add(Console.ReadLine());`	نقرأ الاسم ونضيفه مباشرة إلى آخر القائمة باستخدام Add.
`break;`	break تُنهي هذا الcase وتخرج من switch. في C# يجب أن ينتهي كل case بأمر مثل break حتى لا يكمل التنفيذ إلى الحالة التالية.
`case "2":`	هذا الجزء للاختيار "2" أي عرض الأسماء.
`for (int i = 0; i < names.Count; i++)`	حلقة for تمر على مواقع القائمة من 0 حتى names.Count - 1، وCount هو عدد الأسماء الحالي.
`Console.WriteLine($"{i + 1}. {names[i]}");`	نطبع رقم الاسم i + 1 (لنبدأ العدّ من 1 للمستخدم) ثم الاسم نفسه names[i]، فالقائمة مثل المصفوفة نصل إلى عناصرها بالفهرس بين [ ].

`case "3":`	هذا الجزء للاختيار "3" أي حذف اسم.
`Console.Write("الاسم المراد حذفه: ");`	نطلب من المستخدم كتابة الاسم الذي يريد حذفه.
`if (names.Remove(Console.ReadLine())) Console.WriteLine("تم الحذف");`	الدالة Remove تحاول حذف الاسم وتُرجع true إذا وجدته وحذفته، وfalse إذا لم يكن موجودًا؛ لذلك نضعها مباشرة داخل if ونطبع «تم الحذف» عند النجاح.
`else Console.WriteLine("غير موجود");`	وإذا أرجعت Remove القيمة false، أي أن الاسم ليس في القائمة، نطبع «غير موجود».

تمرين 3: تحدٍّ: كلاس المنتج

تحدٍّ

عرّف كلاس Product (Name, Price, Qty) ودالة Total تُرجع Price × Qty. أنشئ 3 منتجات في List واطبع إجمالي قيمة المخزون.

الحل المقترح:

C#

```
var items = new List<Product>
{
    new Product { Name = "قلم", Price = 0.5, Qty = 100 },
    new Product { Name = "دفتر", Price = 1.2, Qty = 40 },
    new Product { Name = "حقيبة", Price = 8, Qty = 5 }
};

double all = 0;
foreach (var p in items)
{
    Console.WriteLine($"{p.Name}: {p.Total()}");
    all += p.Total();
}
Console.WriteLine($"الإجمالي: {all}");

class Product
{
    public string Name { get; set; }
    public double Price { get; set; }
    public int Qty { get; set; }
    public double Total() => Price * Qty;
}
```

ماذا يفعل هذا الكود؟ برنامج مخزن صغير: يعرّف صنف منتج له اسم وسعر وكمية ودالة تحسب قيمته الكلية، ثم يطبع قيمة كل منتج والإجمالي لكل المنتجات.

`var items = new List<Product>`	ننشئ قائمة عناصرها من النوع Product (صنف معرَّف في آخر الملف)، وvar تجعل #C يستنتج نوع المتغير تلقائيًا.
`{`	الأقواس المعقوصة هنا تجمع المنتجات الأولى التي نضعها في القائمة لحظة إنشائها، مفصولة بفواصل.
`new Product { Name = "قلم", Price = 0.5, Qty = 100 },`	new Product ينشئ كائنًا جديدًا من الصنف Product، ونعطي خصائصه قيمها بين { }: الاسم قلم والسعر 0.5 والكمية 100. والسطران التاليان ينشئان الدفتر والحقيبة بالطريقة نفسها.
`double all = 0;`	متغير عشري all يبدأ بصفر لنجمع فيه قيمة كل المنتجات.
`foreach (var p in items)`	نمر على كل منتج في القائمة، ويحمل المتغير p المنتج الحالي.
`Console.WriteLine($"{p.Name}: {p.Total()}");`	نطبع اسم المنتج ثم نستدعي دالته Total() التي تحسب قيمته الكلية. الأقواس () بعد Total ضرورية لأنها دالة تُستدعى وليست خاصية.
`all += p.Total();`	نضيف قيمة هذا المنتج إلى الإجمالي باستخدام =+ أي all = all + p.Total().
`Console.WriteLine($"الإجمالي: {all}");`	بعد انتهاء الحلقة نطبع مجموع قيم كل المنتجات.
`class Product`	نعرّف الصنف Product، وهو قالب يصف كل منتج: ما يحمله من بيانات (خصائص) وما يستطيع فعله (دوال). تأتي الأصناف بعد أوامر البرنامج الرئيسية في الملف.
`public string Name { get; set; }`	خاصية نصية Name لاسم المنتج؛ public تعني أنها متاحة من خارج الصنف، و{ get; set; } تعني أنها قابلة للقراءة والتغيير.
`public double Price { get; set; }`	خاصية Price لسعر الوحدة، ونوعها double لأن السعر قد يحتوي على كسور مثل 0.5.
`public int Qty { get; set; }`	خاصية Qty للكمية، ونوعها int لأن الكمية عدد صحيح.
`public double Total() ⇒ Price * Qty;`	دالة داخل الصنف اسمها Total تُرجع double. السهم => طريقة مختصرة تعني «أرجِع هذه النتيجة»، بدل كتابة { return Price * Qty; }، فتحسب السعر مضروبًا في الكمية لهذا المنتج.

اختبر نفسك (3 أسئلة)

1. الكلمة void تعني أن الدالة:

لا تُرجع قيمة | تُرجع رقمًا | فارغة من الأوامر

2. لإضافة عنصر إلى List نستخدم:

Push | Add | Insert فقط

3. الكلاس بالنسبة للكائن هو:

نسخة منه | القالب الذي يُصنع منه | متغير عادي

الإجابات: 1) لا تُرجع قيمة · 2) Add · 3) القالب الذي يُصنع منه

الواجب: جهّز تثبيت SQL Server Express وSSMS على جهازك قبل يوم 8 (الرابط في قسم الخطة).

SQL Server اليوم 8 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

SQL Server: إنشاء قاعدة البيانات والاستعلام

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يشرح مفاهيم قاعدة البيانات: الجدول، العمود، الصف، المفتاح الأساسي.
- ينشئ قاعدة بيانات وجدولًا في SQL Server باستخدام SSMS.
- يختار نوع البيانات المناسب لكل عمود.
- يضيف بيانات بـ INSERT.
- يسترجع البيانات بـ SELECT مع WHERE وORDER BY.

توزيع وقت الحصة

0:00 - 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 - 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 - 1:00	استراحة
1:00 - 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 - 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 - 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. ما هي قاعدة البيانات؟

قاعدة البيانات مكان منظم لحفظ البيانات بشكل دائم. البيانات توضع في **جداول**، والجدول له **أعمدة** (الحقول) و**صفوف** (السجلات). **المفتاح الأساسي** Primary Key عمود قيمته فريدة لكل صف، مثل رقم الطالب.

افتح **SSMS** ← اتصل بالخادم `localhost\SQLEXPRESS` ← New Query.

2. إنشاء قاعدة البيانات والجدول

أهم الأنواع: `INT` عدد صحيح، `NVARCHAR(n)` نص يدعم العربية، `DECIMAL(5,2)` عدد عشري، `DATE` تاريخ، `BIT` صح/خطأ.

```sql
CREATE DATABASE TrainingDB;
GO
USE TrainingDB;
GO
```

```
CREATE TABLE Students (
    Id        INT IDENTITY(1,1) PRIMARY KEY,
    FullName  NVARCHAR(100) NOT NULL,
    Email     NVARCHAR(100),
    Age       INT,
    Course    NVARCHAR(50),
    Grade     DECIMAL(5,2),
    JoinDate  DATE DEFAULT GETDATE()
);
```

ماذا يفعل هذا الكود؟ ينشئ هذا الكود قاعدة بيانات جديدة اسمها TrainingDB، ثم ينشئ بداخلها جدولًا اسمه Students لحفظ بيانات الطلاب. عند التشغيل لن ترى بيانات، بل رسالة تفيد بنجاح الأوامر وسيظهر الجدول الفارغ في قاعدة البيانات.

`CREATE DATABASE TrainingDB;`	الأمر CREATE DATABASE ينشئ قاعدة بيانات جديدة فارغة اسمها TrainingDB. قاعدة البيانات مثل خزانة كبيرة سنضع فيها جداول البيانات.
`GO`	الكلمة GO ليست أمر SQL حقيقيًا، بل إشارة لبرنامج SSMS بأن ينفّذ الأوامر التي قبلها كدفعة مستقلة قبل الانتقال لما بعدها. نحتاجها هنا حتى تُنشأ قاعدة البيانات فعلًا قبل أن نحاول استخدامها.
`USE TrainingDB;`	الأمر USE يجعل TrainingDB هي قاعدة البيانات الحالية، فكل الأوامر التالية ستُنفّذ بداخلها وليس في قاعدة بيانات أخرى.
`CREATE TABLE Students (`	الأمر CREATE TABLE ينشئ جدولًا جديدًا اسمه Students. الجدول يشبه جدول Excel: له أعمدة (أنواع المعلومات) وصفوف (كل صف طالب). بين القوسين نكتب الأعمدة واحدًا تلو الآخر مفصولة بفواصل.
`Id  INT IDENTITY(1,1) PRIMARY KEY,`	العمود Id رقم صحيح (INT). الخاصية IDENTITY(1,1) تجعل SQL Server يملؤه تلقائيًا بدءًا من 1 ويزيد 1 مع كل صف جديد، و PRIMARY KEY (المفتاح الأساسي) يعني أن قيمته لا تتكرر ولا تكون فارغة، فيُميّز كل طالب عن غيره.
`FullName  NVARCHAR(100) NOT NULL,`	العمود FullName نصّ من نوع NVARCHAR(100) أي حتى 100 حرف، وحرف N يعني أنه يدعم كل اللغات ومنها العربية. أما NOT NULL فتعني أن هذا العمود إلزامي ولا يمكن ترك الاسم فارغًا.
`Email  NVARCHAR(100),`	العمود Email نصّ حتى 100 حرف لحفظ البريد الإلكتروني. لم نكتب NOT NULL، لذلك يمكن تركه فارغًا (NULL) إذا لم يكن للطالب بريد.
`Age  INT,`	العمود Age يحفظ عمر الطالب كرقم صحيح INT (بدون كسور).
`Course  NVARCHAR(50),`	العمود Course نصّ حتى 50 حرفًا لحفظ اسم الدورة التي يدرسها الطالب.
`Grade  DECIMAL(5,2),`	العمود Grade رقم عشري من نوع DECIMAL(5,2): مجموع خاناته 5 منها خانتان بعد الفاصلة، أي يمكنه حفظ قيم مثل 95.50 وحتى 999.99. نستخدمه لأن الدرجة قد تحتوي كسورًا.

`JoinDate  DATE DEFAULT GETDATE()`	العمود JoinDate من نوع DATE لحفظ تاريخ الانضمام. الكلمة DEFAULT تعني قيمة افتراضية، و GETDATE() تعيد تاريخ اليوم؛ فإن لم نُدخل تاريخًا يُحفظ تاريخ يوم الإضافة تلقائيًا. لاحظ أنه آخر عمود فلا فاصلة بعده.
`);`	القوس المغلق ينهي قائمة الأعمدة، والفاصلة المنقوطة تنهي أمر CREATE TABLE.

IDENTITY(1,1) يجعل SQL Server يولد رقم Id تلقائيًا: 1، 2، 3... ونضع N قبل النص العربي: N'سارة'.

3. إضافة البيانات INSERT

```sql
INSERT INTO Students (FullName, Email, Age, Course, Grade)
VALUES
 (N'سارة أحمد',  'sara@mail.com',  22, N'C#',   95),
 (N'خالد سالم',  'khaled@mail.com',25, N'HTML', 78),
 (N'مريم يوسف',  'maryam@mail.com',20, N'SQL',  88);
```

ماذا يفعل هذا الكود؟ يضيف هذا الكود ثلاثة طلاب إلى جدول Students دفعة واحدة. عند التشغيل تظهر رسالة مثل (3 rows affected) أي تمت إضافة 3 صفوف.

`INSERT INTO Students (FullName, Email, Age, Course, Grade)`	الأمر INSERT INTO يضيف صفوفًا جديدة إلى الجدول Students، وبين القوسين نحدد الأعمدة التي سنعطيها قيمًا. لم نذكر Id لأن IDENTITY يملؤه تلقائيًا، ولا JoinDate لأن له قيمة افتراضية.
`VALUES`	الكلمة VALUES تعني: الآن تأتي القيم. كل مجموعة بين قوسين تمثل صفًا (طالبًا) واحدًا، وترتيب القيم يطابق ترتيب الأعمدة المذكورة أعلاه.
`(N'سارة أحمد',  'sara@mail.com', 22, N'C#',   95),`	هذا السطر والسطران بعده يمثّل كل واحد منها طالبًا. النصوص توضع بين علامتي تنصيص مفردة ''، والبادئة N قبل النص تخبر SQL Server أنه نص Unicode حتى تُحفظ الحروف العربية صحيحة بدل علامات ؟؟؟. الأرقام تُكتب بلا علامات تنصيص، والصفوف تفصل بينها فاصلة.
`(N'مريم يوسف', 'maryam@mail.com',20, N'SQL', 88);`	آخر صف لا تأتي بعده فاصلة، بل فاصلة منقوطة تنهي أمر INSERT كله.

4. استرجاع البيانات SELECT

`*` تعني كل الأعمدة. `WHERE` تصفّي الصفوف، و `ORDER BY` ترتّبها (`DESC` تنازليًا).

```sql
```

```
SELECT * FROM Students;

SELECT FullName, Grade FROM Students;

SELECT * FROM Students WHERE Grade ⩾ 80;

SELECT * FROM Students WHERE Course = N'C#' AND Age < 25;

SELECT FullName, Grade FROM Students ORDER BY Grade DESC;

SELECT * FROM Students WHERE FullName LIKE N'م%';   --يبدأ بحرف م
```

ماذا يفعل هذا الكود؟ مجموعة من استعلامات SELECT لقراءة البيانات من جدول Students بطرق مختلفة: كل البيانات، أعمدة محددة، مع شروط، مع ترتيب، ومع بحث نصي. كل استعلام يعرض نتيجته في جدول مستقل.

`SELECT * FROM Students;`	الأمر SELECT يقرأ البيانات ويعرضها، والنجمة * تعني كل الأعمدة، و FROM Students تحدد الجدول. النتيجة: كل الطلاب بكل معلوماتهم.
`SELECT FullName, Grade FROM Students;`	هنا بدل النجمة كتبنا أسماء الأعمدة التي نريدها فقط، فتظهر النتيجة بعمودين: الاسم والدرجة.
`SELECT * FROM Students WHERE Grade ⩾ 80;`	الكلمة WHERE تضع شرطًا لتصفية الصفوف، فلا يظهر إلا الطلاب الذين درجتهم أكبر من أو تساوي 80.
`SELECT * FROM Students WHERE Course = N'C#' AND Age < 25;`	الكلمة AND تجمع شرطين ويجب أن يتحقق كلاهما: الدورة هي C# والعمر أقل من 25. في SQL نستخدم علامة = واحدة للمقارنة.
`SELECT FullName, Grade FROM Students ORDER BY Grade DESC;`	الكلمة ORDER BY ترتّب النتائج حسب عمود معيّن، و DESC تعني تنازليًا (من الأكبر إلى الأصغر)، فيظهر صاحب أعلى درجة أولًا. بدونها يكون الترتيب تصاعديًا (ASC).
`SELECT * FROM Students WHERE FullName LIKE N'م%';   --يبدأ بحرف م`	الكلمة LIKE تبحث عن نمط في النص، والرمز % يعني: أي عدد من الحروف (أو لا شيء). إذن N'م%' تعني كل اسم يبدأ بحرف م. والجزء بعد -- تعليق للتوضيح يتجاهله SQL Server.

التمارين العملية

تمرين 1: أنشئ القاعدة والجدول

سهل

أنشئ قاعدة TrainingDB وجدول Students كما في الشرح، ثم أدخل بيانات 6 طلاب (زملاؤك في الدورة).

الحل المقترح:

```
SQL
```

```
INSERT INTO Students (FullName, Email, Age, Course, Grade) VALUES
 (N'سارة', 's@mail.com', 22, N'C#', 95),
 (N'خالد', 'k@mail.com', 25, N'HTML', 78),
 (N'مريم', 'm@mail.com', 20, N'SQL', 88),
 (N'علي', 'a@mail.com', 23, N'C#', 55),
 (N'نورة', 'n@mail.com', 21, N'HTML', 91),
 (N'يوسف', 'y@mail.com', 24, N'SQL', 67);
```

ماذا يفعل هذا الكود؟ حل التمرين: إضافة ستة طلاب إلى جدول Students بأمر INSERT واحد. عند التشغيل تظهر رسالة (6 rows affected).

`INSERT INTO Students (FullName, Email, Age, Course, Grade) VALUES`	الأمر INSERT INTO يضيف صفوفًا إلى الجدول Students في الأعمدة المذكورة بين القوسين، ثم تأتي VALUES وبعدها القيم. لا نكتب Id لأنه يُولّد تلقائيًا.
`(N'سارة', 's@mail.com', 22, N'C#', 95),`	هذا السطر والسطران بعده ثلاثة طلاب، كل قوسين صف واحد بالترتيب: الاسم، البريد، العمر، الدورة، الدرجة. البادئة N قبل النصوص العربية ضرورية لحفظ الحروف العربية بشكل صحيح.
`(N'علي', 'a@mail.com', 23, N'C#', 55),`	هذا السطر والسطران بعده ثلاثة طلاب آخرون بنفس الترتيب. تفصل الفاصلة بين الصفوف، والصف الأخير ينتهي بفاصلة منقوطة لإنهاء الأمر.

تمرين 2: استعلامات

متوسط

اكتب استعلامات تعرض: (أ) أسماء الطلاب ودرجاتهم مرتبة من الأعلى، (ب) طلاب دورة C# فقط، (ج) الطلاب الراسبين (أقل من 60)، (د) الطلاب بين 20 و23 سنة.

الحل المقترح:

SQL

```
-- أ
 SELECT FullName, Grade FROM Students ORDER BY Grade DESC;
-- ب
 SELECT * FROM Students WHERE Course = N'C#';
-- ج
 SELECT * FROM Students WHERE Grade < 60;
-- د
 SELECT * FROM Students WHERE Age BETWEEN 20 AND 23;
```

ماذا يفعل هذا الكود؟ حل التمرين: أربعة استعلامات SELECT، كل واحد يجيب عن سؤال (أ، ب، ج، د) ويعرض نتيجته في جدول مستقل.

`-- أ`	السطر الذي يبدأ بـ -- تعليق، أي ملاحظة للقارئ لا ينفذها SQL Server. استخدمناه هنا لترقيم أجزاء التمرين.
`SELECT FullName, Grade FROM Students ORDER BY Grade DESC;`	يعرض الاسم والدرجة لكل الطلاب مرتّبين بـ ORDER BY حسب الدرجة، و DESC تعني من الأعلى إلى الأدنى.
`SELECT * FROM Students WHERE Course = N'C#';`	يعرض كل أعمدة الطلاب الذين دورتهم C# فقط، لأن WHERE تُبقي الصفوف التي يتحقق فيها الشرط. استخدمنا N'' لأنها مقارنة بعمود من نوع NVARCHAR.
`SELECT * FROM Students WHERE Grade < 60;`	يعرض الطلاب الذين درجتهم أقل من 60، أي الراسبين.
`SELECT * FROM Students WHERE Age BETWEEN 20 AND 23;`	الكلمة BETWEEN تختار القيم الواقعة في مدى معيّن، وهي تشمل الحدّين؛ أي الأعمار 20 و21 و22 و23. هي اختصار لـ Age >= 20 AND Age <= 23.

تمرين 3: تحدٍّ: جدول الدورات

تحدٍّ

أنشئ جدول Courses (Id, Title, Hours) وأدخل 3 دورات. ثم اعرض الدورات التي تزيد ساعاتها على 6.

الحل المقترح:

```SQL
CREATE TABLE Courses (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Title NVARCHAR(50) NOT NULL,
    Hours INT
);
INSERT INTO Courses (Title, Hours) VALUES (N'HTML', 6), (N'C#', 8), (N'SQL', 4);
SELECT * FROM Courses WHERE Hours > 6;
```

ماذا يفعل هذا الكود؟ حل التمرين: إنشاء جدول جديد للدورات Courses، وإضافة ثلاث دورات إليه، ثم عرض الدورات التي تزيد ساعاتها على 6. النتيجة ستكون صفًا واحدًا هو دورة C# (8 ساعات).

`CREATE TABLE Courses (`	الأمر CREATE TABLE ينشئ جدولًا جديدًا اسمه Courses، وبين القوسين نعرّف أعمدته.
`Id INT IDENTITY(1,1) PRIMARY KEY,`	العمود Id رقم صحيح يُملأ تلقائيًا (IDENTITY يبدأ من 1 ويزيد 1)، وهو المفتاح الأساسي PRIMARY KEY الذي يميّز كل دورة ولا يتكرر.
`Title NVARCHAR(50) NOT NULL,`	العمود Title اسم الدورة، نص يدعم العربية حتى 50 حرفًا، و NOT NULL تجعله إلزاميًا.

`Hours INT`	العمود Hours عدد ساعات الدورة كرقم صحيح. هو آخر عمود لذلك لا فاصلة بعده.
`);`	القوس المغلق ينهي تعريف الأعمدة، والفاصلة المنقوطة تنهي أمر إنشاء الجدول.
`INSERT INTO Courses (Title, Hours) VALUES (N'HTML', 6), (N'C#', 8), (N'SQL', 4);`	يضيف ثلاث دورات في سطر واحد، كل قوسين صف: اسم الدورة وعدد ساعاتها. العمود Id لا نكتبه لأنه يُولّد تلقائيًا.
`SELECT * FROM Courses WHERE Hours > 6;`	يعرض الدورات التي ساعاتها أكبر من 6 فقط. لاحظ أن > تعني أكبر تمامًا، لذلك دورة HTML (6 ساعات) لا تظهر.

اختبر نفسك (3 أسئلة)

1. العمود الذي يميز كل صف بقيمة فريدة هو:

Index | Primary Key | Foreign Key

2. أي أمر يضيف صفًا جديدًا؟

CREATE | INSERT | SELECT

3. لترتيب النتائج تنازليًا نكتب:

WHERE DESC | SORT Grade DOWN | ORDER BY Grade DESC

الإجابات: 1) Primary Key · 2) INSERT · 3) ORDER BY Grade DESC

الواجب: أنشئ قاعدة بيانات مشروعك النهائي وجدولها الرئيسي، وأدخل 5 صفوف تجريبية.

SQL Server اليوم 9 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

تعديل البيانات والتجميع وربط #C بـ SQL Server

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يعدّل البيانات بـ UPDATE ويحذفها بـ DELETE مع WHERE دائمًا.
- يحسب الإحصائيات بـ COUNT وAVG وMAX وMIN وGROUP BY.
- يتصل بقاعدة البيانات من برنامج #C باستخدام Microsoft.Data.SqlClient.
- يقرأ الصفوف بـ SqlDataReader ويضيف صفوفًا باستعلام ذي باراميترات.

توزيع وقت الحصة

0:00 - 0:10	مراجعة SELECT وWHERE
0:10 - 0:35	UPDATE وDELETE ودوال التجميع
0:35 - 1:05	ربط #C بـ SQL Server: القراءة والإضافة
1:05 - 1:15	استراحة
1:15 - 1:40	تمارين: برنامج #C متصل بالقاعدة
1:40 - 2:00	العمل على المشروع النهائي بإشراف المدرب

شرح الدرس

1. التعديل والحذف

```sql
UPDATE Students SET Grade = 82 WHERE Id = 2;

UPDATE Students SET Course = N'C#', Age = 24 WHERE FullName = N'علي';

DELETE FROM Students WHERE Id = 6;
```

ماذا يفعل هذا الكود؟ أوامر لتعديل بيانات موجودة (UPDATE) وحذف صف (DELETE) من جدول Students. عند التشغيل تظهر رسالة بعدد الصفوف التي تأثرت مثل (1 row affected).

```
UPDATE Students SET Grade = 82
WHERE Id = 2;
```

الأمر UPDATE يعدّل بيانات موجودة، و SET تحدد العمود وقيمته الجديدة، و WHERE تحدد أي صف سيتعدّل (الطالب صاحب Id = 2). تحذير: لو نسيت WHERE ستتغير درجة كل الطلاب في الجدول!

```
UPDATE Students SET Course =
N'C#', Age = 24 WHERE FullName =
N'علي';
```

يمكن تعديل أكثر من عمود في أمر واحد بفصلها بفاصلة بعد SET. هنا نغيّر دورة وعمر الطالب الذي اسمه علي، والبادئة N ضرورية لأن الاسم عربي.

`DELETE FROM Students WHERE Id = 6;`	الأمر DELETE FROM يحذف صفوفًا من الجدول، و WHERE تحدد الطالب صاحب Id = 6 فقط. تحذير مهم: DELETE بدون WHERE يحذف كل الصفوف، ولا يوجد زر تراجع، لذلك تأكد دائمًا من الشرط.

تحذير مهم: UPDATE أو DELETE بدون WHERE يغيّر أو يحذف كل صفوف الجدول! اكتب SELECT بنفس الشرط أولًا لتتأكد.

2. دوال التجميع

```SQL
SELECT COUNT(*) AS StudentsCount FROM Students;
SELECT AVG(Grade) AS Average, MAX(Grade) AS Highest, MIN(Grade) AS Lowest FROM Students;

-- عدد الطلاب ومتوسطهم لكل دورة
SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg
FROM Students
GROUP BY Course;
```

ماذا يفعل هذا الكود؟ يستخدم الدوال التجميعية لحساب إحصائيات عن الطلاب: العدد، والمتوسط، وأعلى وأدنى درجة، ثم نفس الإحصائيات لكل دورة على حدة باستخدام GROUP BY.

`SELECT COUNT(*) AS StudentsCount FROM Students;`	الدالة COUNT(*) تعدّ صفوف الجدول وتعيد رقمًا واحدًا هو عدد الطلاب. الكلمة AS تعطي عمود النتيجة اسمًا مستعارًا (StudentsCount) ليظهر بعنوان واضح بدل عمود بلا اسم.
`SELECT AVG(Grade) AS Average, MAX(Grade) AS Highest, MIN(Grade) AS Lowest FROM Students;`	الدالة AVG تحسب متوسط الدرجات، و MAX تعيد أعلى درجة، و MIN أدنى درجة. النتيجة صف واحد فيه ثلاثة أعمدة، ولكل عمود اسم أعطيناه بـ AS.
`-- عدد الطلاب ومتوسطهم لكل دورة`	تعليق يبدأ بـ -- لشرح الاستعلام التالي، ولا ينفذه SQL Server.
`SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg`	نختار اسم الدورة، وعدد الطلاب (Total)، ومتوسط الدرجات (Avg). هذا الاستعلام مكتوب على عدة أسطر لسهولة القراءة، وهو أمر واحد ينتهي عند الفاصلة المنقوطة.
`FROM Students`	نأخذ البيانات من جدول Students.
`GROUP BY Course;`	الكلمة GROUP BY تقسّم الطلاب إلى مجموعات حسب الدورة، ثم تُحسب COUNT و AVG لكل مجموعة على حدة. النتيجة صف لكل دورة. قاعدة: أي عمود عادي في SELECT (مثل Course) يجب أن يكون مذكورًا في GROUP BY.

3. ربط C# بـ SQL Server

1) في Visual Studio: Project ← Manage NuGet Packages ← ابحث عن `Microsoft.Data.SqlClient` وثبّته.

2) جهّز **نص الاتصال** Connection String.

3) افتح الاتصال، نفذ الأمر، اقرأ النتائج.

C#

```
using Microsoft.Data.SqlClient;

string cs =
@"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCerti
ficate=True;";

using var conn = new SqlConnection(cs);
conn.Open();

var cmd = new SqlCommand("SELECT Id, FullName, Grade FROM Students ORDER BY Grade DESC",
conn);
using var reader = cmd.ExecuteReader();

while (reader.Read())
{
    Console.WriteLine($"{reader["Id"]}. {reader["FullName"]} - {reader["Grade"]}");
}
```

ماذا يفعل هذا الكود؟ برنامج C# يتصل بقاعدة البيانات TrainingDB ويقرأ الطلاب مرتبين حسب الدرجة من الأعلى، ثم يطبع كل طالب في سطر بالشكل: الرقم. الاسم - الدرجة.

`using Microsoft.Data.SqlClient;`	توجيه using في أول الملف يستورد مكتبة Microsoft.Data.SqlClient حتى نستخدم أصنافها مثل SqlConnection و SqlCommand بأسمائها المختصرة. يجب أولًا إضافة حزمة NuGet بنفس الاسم إلى المشروع.
`string cs = @"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCertificate=True;";`	هذا نص الاتصال (connection string) ويخبر البرنامج كيف يصل لقاعدة البيانات: Server هو اسم الخادم (localhost\SQLEXPRESS أي نسخة SQLEXPRESS على جهازك)، و Database اسم القاعدة، و Trusted_Connection=True يعني الدخول بحساب Windows بلا كلمة مرور، و TrustServerCertificate=True يقبل شهادة الأمان المحلية للخادم دون تحقق. علامة @ قبل النص تجعل \ حرفًا عاديًا وليس بداية رمز خاص.
`using var conn = new SqlConnection(cs);`	ننشئ كائن اتصال SqlConnection باستخدام نص الاتصال. كلمة using var (تختلف عن توجيه using في أول الملف) تضمن إغلاق الاتصال وتحرير موارده تلقائيًا عند انتهاء البرنامج أو الجزء الذي هو فيه، حتى لو حدث خطأ.

`conn.Open();`	الدالة Open تفتح الاتصال فعليًا بالخادم. لا يمكن تنفيذ أي أمر قبل فتح الاتصال، وإن كانت بيانات الاتصال خاطئة سيظهر الخطأ هنا.
`var cmd = new SqlCommand("SELECT Id, FullName, Grade FROM Students ORDER BY Grade DESC", conn);`	الكائن SqlCommand يحمل أمر SQL الذي نريد تنفيذه (هنا استعلام SELECT مرتّب تنازليًا بالدرجة) والاتصال الذي سيُنفّذ عليه.
`using var reader = cmd.ExecuteReader();`	الدالة ExecuteReader تنفّذ الاستعلام وتعيد قارئًا (SqlDataReader) نقرأ به النتائج صفًا صفًا. نستخدم using var لإغلاق القارئ تلقائيًا عند الانتهاء.
`while (reader.Read())`	الدالة Read تنتقل إلى الصف التالي وتعيد true إن وُجد صف، و false عندما تنتهي الصفوف. لذلك نضعها في حلقة while لتكرار القراءة حتى آخر صف.
`{`	القوس المعقوص { يبدأ جسم الحلقة، أي الأوامر التي تتكرر لكل صف، والقوس } ينهيه.
`Console.WriteLine($" {reader["Id"]}. {reader["FullName"]} - {reader["Grade"]}");`	نطبع الصف الحالي. reader["Id"] يجلب قيمة العمود Id من الصف الحالي بكتابة اسم العمود، وكذلك FullName و Grade. علامة $ قبل النص تسمح بوضع القيم داخل { } مباشرة في النص.

4. الإضافة باستخدام البارامترات

لا تدمج مدخلات المستخدم مباشرة في نص الاستعلام، بل استخدم **بارامترات** مثل `@name` . هذا يحمي من ثغرة SQL Injection ويتعامل مع العربية بشكل صحيح.

C#

```
Console.Write("الاسم: ");
string name = Console.ReadLine();
Console.Write("الدرجة: ");
decimal grade = decimal.Parse(Console.ReadLine());

using var conn = new SqlConnection(cs);
conn.Open();
var cmd = new SqlCommand(
    "INSERT INTO Students (FullName, Grade) VALUES (@name, @grade)", conn);
cmd.Parameters.AddWithValue("@name", name);
cmd.Parameters.AddWithValue("@grade", grade);

int rows = cmd.ExecuteNonQuery();
Console.WriteLine($"تمت إضافة {rows} صف");
```

ماذا يفعل هذا الكود؟ برنامج يطلب من المستخدم اسم طالب ودرجته، ثم يضيفه إلى جدول Students بأمان باستخدام المعاملات (parameters)، ويطبع عدد الصفوف التي أضيفت: تمت إضافة 1 صف.

`Console.Write("الاسم: ");`	نطبع سؤالًا للمستخدم. Write تبقي المؤشر على نفس السطر ليكتب المستخدم الإجابة بجانب السؤال.
`string name = Console.ReadLine();`	ReadLine تنتظر المستخدم حتى يكتب نصًا ويضغط Enter، ثم نحفظ ما كتبه في المتغير name.
`Console.Write("الدرجة: ");`	نطلب من المستخدم إدخال الدرجة.
`decimal grade = decimal.Parse(Console.ReadLine());`	ما يكتبه المستخدم يصل دائمًا كنص، لذلك نحوّله إلى رقم عشري بـ decimal.Parse. إن كتب المستخدم شيئًا ليس رقمًا سيتوقف البرنامج بخطأ.
`using var conn = new SqlConnection(cs);`	ننشئ اتصالًا بقاعدة البيانات باستخدام نص الاتصال cs المعرّف في المثال السابق، و using var تغلقه تلقائيًا عند الانتهاء.
`conn.Open();`	نفتح الاتصال بالخادم قبل تنفيذ أي أمر.
`var cmd = new SqlCommand(`	ننشئ أمر SqlCommand، وقد كتبناه على سطرين لأن النص طويل.
`"INSERT INTO Students (FullName, Grade) VALUES (@name, @grade)", conn);`	هذا أمر الإضافة، لكن بدل وضع القيم مباشرة كتبنا @name و @grade، وهي معاملات (parameters): أماكن محجوزة ستُملأ بالقيم لاحقًا بشكل منفصل عن نص الأمر.
`cmd.Parameters.AddWithValue("@name", name);`	AddWithValue تربط المعامل @name بقيمة المتغير name. القيمة تُرسل للخادم كبيانات فقط وليست كجزء من أمر SQL، فلو كتب المستخدم نصًا خبيثًا لن يُنفّذ كأمر، وهذا يحمينا من هجوم SQL Injection (حقن أوامر ضارة).
`cmd.Parameters.AddWithValue("@grade", grade);`	نربط المعامل @grade بقيمة المتغير grade بنفس الطريقة الآمنة. لا تبنِ أوامر SQL أبدًا بلصق مدخلات المستخدم داخل النص.
`int rows = cmd.ExecuteNonQuery();`	ExecuteNonQuery تستخدم للأوامر التي لا تعيد جدول نتائج مثل INSERT و UPDATE و DELETE، وتعيد عدد الصفوف التي تأثرت، فنحفظه في rows.
`Console.WriteLine($"تمت إضافة {rows} صف");`	نطبع رسالة تأكيد فيها عدد الصفوف المضافة، وسيكون 1 عند النجاح.

ExecuteReader للقراءة (SELECT)، وExecuteNonQuery للإضافة والتعديل والحذف، وExecuteScalar لقيمة واحدة مثل COUNT.

التمارين العملية

تمرين 1: تعديل وإحصاء

سهل

ارفع درجة كل طلاب دورة HTML بمقدار 5، ثم اعرض عدد الطلاب ومتوسط الدرجات لكل دورة.

الحل المقترح:

```SQL
UPDATE Students SET Grade = Grade + 5 WHERE Course = N'HTML';

SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg
FROM Students GROUP BY Course;
```

ماذا يفعل هذا الكود؟ حل التمرين: زيادة درجة كل طلاب دورة HTML بمقدار 5، ثم عرض عدد الطلاب ومتوسط الدرجات لكل دورة للتأكد من التغيير.

`UPDATE Students SET Grade = Grade + 5 WHERE Course = N'HTML';`	الأمر UPDATE مع SET Grade = Grade + 5 يأخذ الدرجة الحالية لكل صف ويضيف لها 5. الشرط WHERE يحصر التعديل في طلاب دورة HTML فقط؛ وبدونه ستزيد درجات الجميع.
`SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg`	نختار اسم الدورة، وعدد طلابها بـ COUNT(*)، ومتوسط درجاتها بـ AVG، ونعطي الأعمدة أسماء واضحة بـ AS.
`FROM Students GROUP BY Course;`	نأخذ البيانات من Students ونجمّعها بـ GROUP BY حسب الدورة، فتظهر نتيجة واحدة لكل دورة، وسترى أن متوسط HTML ارتفع بمقدار 5.

تمرين 2: عرض الطلاب من C#

متوسط

اكتب برنامج C# يتصل بـ TrainingDB ويعرض كل الطلاب الناجحين (60 فأكثر) مع دوراتهم.

الحل المقترح:

```C#
using Microsoft.Data.SqlClient;

string cs =
@"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCe
rtificate=True;";
using var conn = new SqlConnection(cs);
conn.Open();

var cmd = new SqlCommand("SELECT FullName, Course, Grade FROM Students WHERE Grade ⩾
60", conn);
using var r = cmd.ExecuteReader();
while (r.Read())
    Console.WriteLine($"{r["FullName"]} | {r["Course"]} | {r["Grade"]}");
```

ماذا يفعل هذا الكود؟ حل التمرين: برنامج #C يتصل بقاعدة البيانات ويطبع الطلاب الناجحين (درجتهم 60 فأكثر) بالشكل: الاسم | الدورة | الدرجة.

`using Microsoft.Data.SqlClient;`	توجيه using يستورد مكتبة Microsoft.Data.SqlClient لنستخدم أصناف الاتصال بـ SQL Server مثل SqlConnection و SqlCommand.
`string cs = @"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCertificate=True;";`	نص الاتصال: Server اسم الخادم (نسخة SQLEXPRESS على جهازك)، و Database اسم القاعدة، و Trusted_Connection=True للدخول بحساب Windows، و TrustServerCertificate=True لقبول شهادة الخادم المحلي. علامة @ تجعل الشرطة المائلة \ حرفًا عاديًا داخل النص.
`using var conn = new SqlConnection(cs);`	ننشئ كائن الاتصال، و using var تضمن إغلاقه تلقائيًا عند الانتهاء.
`conn.Open();`	نفتح الاتصال بالخادم فعليًا.
`var cmd = new SqlCommand("SELECT FullName, Course, Grade FROM Students WHERE Grade ⩾ 60", conn);`	نجهّز أمر SELECT يجلب الاسم والدورة والدرجة للطلاب الذين درجتهم 60 أو أكثر، ونربطه بالاتصال conn.
`using var r = cmd.ExecuteReader();`	ExecuteReader تنفّذ الاستعلام وتعيد قارئًا سمّيناه r لنقرأ النتائج صفًا صفًا.
`while (r.Read())`	r.Read() تنتقل للصف التالي وتعيد true ما دام هناك صفوف، فتستمر الحلقة حتى آخر صف. لم نضع { } لأن جسم الحلقة سطر واحد فقط.
`Console.WriteLine($" {r["FullName"]} | {r["Course"]} | {r["Grade"]}");`	نطبع قيم أعمدة الصف الحالي بكتابة اسم كل عمود بين القوسين المربعين، مثل r["FullName"]، ونفصل بينها بالرمز |.

تمرين 3: تحدٍّ: عدد الطلاب ومتوسطهم من #C

تحدٍّ

استخدم ExecuteScalar لطباعة عدد الطلاب، ثم متوسط الدرجات بخانة عشرية واحدة.

الحل المقترح:

C#

```
using var conn = new SqlConnection(cs);
conn.Open();

int count = (int)new SqlCommand("SELECT COUNT(*) FROM Students", conn).ExecuteScalar();
```

```
decimal avg = (decimal)new SqlCommand("SELECT AVG(Grade) FROM Students",
conn).ExecuteScalar();

Console.WriteLine($"عدد الطلاب: {count}");
Console.WriteLine($"المتوسط: {avg:F1}");
```

ماذا يفعل هذا الكود؟ حل التمرين: برنامج يحسب عدد الطلاب ومتوسط درجاتهم من قاعدة البيانات باستخدام ExecuteScalar، ثم يطبعهما مع تقريب المتوسط لرقم عشري واحد.

`using var conn = new SqlConnection(cs);`	ننشئ اتصالًا بقاعدة البيانات باستخدام نص الاتصال cs (المعرّف سابقًا)، و using var تغلقه تلقائيًا.
`conn.Open();`	نفتح الاتصال قبل تنفيذ الأوامر.
`int count = (int)new SqlCommand("SELECT COUNT(*) FROM Students", conn).ExecuteScalar();`	ExecuteScalar تنفّذ الاستعلام وتعيد قيمة واحدة فقط (أول خانة في النتيجة)، وهي مناسبة لـ COUNT. لكنها تعيد النوع العام object، لذلك نحوّلها بـ (int) إلى رقم صحيح لنحفظها في count.
`decimal avg = (decimal)new SqlCommand("SELECT AVG(Grade) FROM Students", conn).ExecuteScalar();`	بنفس الطريقة نجلب متوسط الدرجات ونحوّله بـ (decimal) لأن العمود Grade عشري. ملاحظة: إن كان الجدول فارغًا فإن AVG تعيد NULL ويفشل هذا التحويل.
`Console.WriteLine($"عدد الطلاب: {count}");`	نطبع عدد الطلاب داخل الرسالة.
`Console.WriteLine($"المتوسط: {avg:F1}");`	نطبع المتوسط، والتنسيق F1: يعرض الرقم برقم واحد بعد الفاصلة العشرية (مثل 83.5) بدل خانات كثيرة.

اختبر نفسك (3 أسئلة)

1. ماذا يحدث عند تنفيذ DELETE FROM Students بدون WHERE؟

لا شيء | يُحذف أول صف | تُحذف كل الصفوف

2. أي دالة تحسب المتوسط؟

SUM | AVG | COUNT

3. لتنفيذ INSERT من C# نستخدم:

ExecuteReader | ExecuteNonQuery | ReadLine

الإجابات: 1) تُحذف كل الصفوف · 2) ExecuteNonQuery (3 · AVG

الواجب: أكمل المشروع النهائي وجهّز عرضًا مدته 5 دقائق.

المشروع اليوم 10 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

المشروع النهائي: الإكمال والعرض والاختبار النهائي

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يدمج HTML وC# وSQL Server في مشروع واحد يعمل.
- يشرح الكود الذي كتبه ويجيب عن أسئلة حوله.
- يقيّم ما تعلمه ويعرف الخطوة التالية في رحلته.

توزيع وقت الحصة

0:00 – 0:30	اللمسات الأخيرة على المشاريع وحل المشكلات مع المدرب
0:30 – 1:15	عرض المشاريع: 6 متدربين × 7 دقائق (5 عرض + 2 أسئلة)
1:15 – 1:25	استراحة
1:25 – 1:50	الاختبار النهائي (20 درجة)
1:50 – 2:00	إعلان النتائج والتغذية الراجعة والخطوات التالية

شرح الدرس

1. قائمة التحقق قبل العرض

- صفحة HTML تفتح وتعرض اسم النظام والنموذج والجدول بشكل منسق.
- قاعدة البيانات موجودة وفيها 5 صفوف على الأقل.
- برنامج C# يعمل بدون أخطاء، وكل خيارات القائمة تعمل.
- جربت إدخال قيمة خاطئة (حروف بدل رقم) ولم يتوقف البرنامج.
- جهزت 3 نقاط تريد أن تقولها: الفكرة، أصعب جزء، ماذا ستضيف لاحقًا.

2. طريقة العرض (5 دقائق)

1. الفكرة ولمن هذا النظام (30 ثانية).
2. صفحة HTML (دقيقة).
3. قاعدة البيانات وتصميم الجدول في SSMS (دقيقة).
4. تشغيل برنامج C# وتجربة: إضافة، عرض، تعديل، حذف، إحصائيات (دقيقتان).
5. ما تعلمته وأصعب مشكلة واجهتها (30 ثانية).

3. ماذا بعد الدورة؟

الخطوات المقترحة: JavaScript لجعل الصفحات تفاعلية، ثم ASP.NET Core لبناء مواقع تتصل بـ SQL Server مباشرة، ثم Entity Framework Core. المهم الآن: اكتب كودًا كل يوم ولو 30 دقيقة.

اختبر نفسك (3 أسئلة)

1. لحماية الاستعلام من SQL Injection نستخدم:

البارامترات | تكبير الخط | حلقة for

2. أي لغة تُستخدم لبناء واجهة صفحة الويب؟

SQL | HTML | C#

3. List أفضل من المصفوفة عندما:

يكون عدد العناصر متغيرًا | نريد رقمًا واحدًا | نكتب HTML

الإجابات: 1) البارامترات · 2) HTML · 3) يكون عدد العناصر متغيرًا

الواجب: مبروك إتمام الدورة!